```
// ==================== [第 1 页 / 共 60 页] 前端交互与视图组件模块 ====================
import React, { useState, useEffect, useRef } from 'react';
import { Target, Download, Zap, RefreshCw, ShoppingCart, Activity, FileText } from 'lucide-react';
import { apiClient, errorMessage } from '../api/client';
import { useAppStore } from '../store/useAppStore';
import { useNavigate } from 'react-router-dom';
interface ResiduePoint {
  idx: number;
  residue: string;
  angle_deg: number;
  cx: number;
  cy: number;
  category: string;
  color: string;
}
interface HelicalWheelData {
  sequence: string;
  length: number;
  residues: ResiduePoint[];
  hydrophobic_moment: number;
  moment_angle_deg: number;
  net_charge: number;
  mean_hydrophobicity: number;
}
export default function HelicalWheelStudio() {
  const { activeSequence, addToCart, setActivePeptide } = useAppStore();
  const [seq, setSeq] = useState(activeSequence);
  const [loading, setLoading] = useState(false);
  const [wheelData, setWheelData] = useState<HelicalWheelData | null>(null);
  const [errorMsg, setErrorMsg] = useState('');
  const svgRef = useRef<SVGSVGElement | null>(null);
  const navigate = useNavigate();
  const handleFetchWheel = async (targetSeq: string) => {
    const cleaned = targetSeq.trim().replace(/\s/g, '').toUpperCase();
    if (!cleaned) {
      setErrorMsg('请输入有效的多肽序列');
      return;
    }
    setErrorMsg('');
    setLoading(true);
    try {
      const res = await apiClient.post<HelicalWheelData>('/api/helical_wheel', {
        sequence: cleaned
      });
      const raw = res.data as any;
      setWheelData({ ...raw, residues: raw.residues.map((r: any) => {
        const radius = 120 - 30 * Math.floor((r.pos - 1) / 18);
        const angle = r.angle_deg * Math.PI / 180;
        return { ...r, idx: r.pos - 1, residue: r.aa, cx: 160 + radius * Math.sin(angle), cy: 160 - radius *
Math.cos(an
      }) });
    } catch (err: any) {
      setWheelData(null);
      setErrorMsg(errorMessage(err));
```

```
// ==================== [第 2 页 / 共 60 页] 前端交互与视图组件模块 ====================
    } finally {
      setLoading(false);
    }
  };
  useEffect(() => {
    if (activeSequence) {
      // oxlint-disable-next-line react/set-state-in-effect
      setSeq(activeSequence);
      handleFetchWheel(activeSequence);
    }
  }, [activeSequence]);
  const handleExportPNG = () => {
    if (!svgRef.current) return;
    const svgElement = svgRef.current;
    const svgString = new XMLSerializer().serializeToString(svgElement);
    const canvas = document.createElement('canvas');
    canvas.width = 640;
    canvas.height = 640;
    const ctx = canvas.getContext('2d');
    if (!ctx) return;
    const img = new Image();
    const svgBlob = new Blob([svgString], { type: 'image/svg+xml;charset=utf-8' });
    const url = URL.createObjectURL(svgBlob);
    img.onload = () => {
      ctx.fillStyle = '#ffffff';
      ctx.fillRect(0, 0, canvas.width, canvas.height);
      ctx.drawImage(img, 0, 0, 640, 640);
      const pngUrl = canvas.toDataURL('image/png');
      const downloadLink = document.createElement('a');
      downloadLink.href = pngUrl;
      downloadLink.download = `HelicalWheel_${seq.slice(0, 8)}_${Date.now()}.png`;
      downloadLink.click();
      URL.revokeObjectURL(url);
    };
    img.src = url;
  };
  // Eisenberg moment vector endpoints
  const cx0 = 160;
  const cy0 = 160;
  const momentLen = wheelData ? Math.min(100, Math.max(25, wheelData.hydrophobic_moment * 180)) : 50;
  const momentRad = wheelData ? (wheelData.moment_angle_deg * Math.PI) / 180 : 0;
  const arrowX = cx0 + momentLen * Math.sin(momentRad);
  const arrowY = cy0 - momentLen * Math.cos(momentRad);
  return (
    <div className="flex flex-col gap-6">
      {/* Studio Header */}
      <div className="border-b border-slate-200 dark:border-slate-800 pb-4">
        <h2 className="text-xl font-bold text-slate-800 dark:text-slate-100 flex items-center gap-2">
          <Zap className="text-amber-500" size={22} />
          螺旋轮构象与两亲性投影室 (Schiffer-Edmundson Helical Wheel Studio)
        </h2>
        <p className="text-xs text-slate-500 mt-1">
```

```
// ==================== [第 3 页 / 共 60 页] 前端交互与视图组件模块 ====================
            按每残基 100° 的理想螺旋假设绘制理化属性投影；每 18 个残基换一显示环以避免重叠，径向层仅用于排版，不代表真实结构或膜作用机制。
          </p>
        </div>
        <div className="grid grid-cols-1 lg:grid-cols-12 gap-6">
          {/* Left Controls & Properties */}
          <div className="lg:col-span-5 flex flex-col gap-4">
            <div className="lab-panel p-4 flex flex-col gap-3">
              <span className="lab-label">输入分析序列</span>
              <div className="flex gap-2">
                <input
                  type="text"
                  value={seq}
                  onChange={(e) => setSeq(e.target.value)}
                  placeholder="输入多肽序列 (例如 GIGAVLKVLTTGLPALISWIKRKRQQ)"
                  className="lab-input font-mono text-xs uppercase flex-1"
                />
                <button
                  type="button"
                  onClick={() => handleFetchWheel(seq)}
                  disabled={loading}
                  className="btn-lab-primary px-3 py-2 text-xs flex items-center gap-1.5 whitespace-nowrap"
                >
                  {loading ? <RefreshCw size={14} className="animate-spin" /> : <Target size={14} />}
                  <span>投影 (Project)</span>
                </button>
              </div>
              {errorMsg && (
                <div className="text-xs text-rose-600 bg-rose-50 dark:bg-rose-950/40 p-2.5 rounded border border-rose-200
                  {errorMsg}
                </div>
              )}
            </div>
            {/* Physicochemical Property Cards */}
            {wheelData && (
              <div className="lab-panel p-4 flex flex-col gap-3">
                <span className="lab-label">生物物理特征计算矩阵</span>
                <div className="grid grid-cols-2 gap-2 text-xs">
                  <div className="p-2.5 rounded bg-slate-50 dark:bg-slate-900 border border-slate-200 dark:border-slate-80
                    <div className="text-[11px] text-slate-500">序列全长 (Length)</div>
                    <div className="text-base font-bold font-mono text-slate-800 dark:text-slate-200">{wheelData.length} A
                  </div>
                  <div className="p-2.5 rounded bg-slate-50 dark:bg-slate-900 border border-slate-200 dark:border-slate-80
                    <div className="text-[11px] text-slate-500">净电荷 (pH 7.4)</div>
                    <div className={`text-base font-bold font-mono ${wheelData.net_charge >= 0 ? 'text-blue-600 dark:text-
                      {wheelData.net_charge > 0 ? `+${wheelData.net_charge.toFixed(1)}` : wheelData.net_charge.toFixed(1)}
                    </div>
                  </div>
                  <div className="p-2.5 rounded bg-slate-50 dark:bg-slate-900 border border-slate-200 dark:border-slate-80
                    <div className="text-[11px] text-slate-500">两亲性力矩 (Eisenberg μH)</div>
                    <div className="text-base font-bold font-mono text-amber-600 dark:text-amber-400">
                      {wheelData.hydrophobic_moment.toFixed(3)}
                    </div>
```

```
// ==================== [第 4 页 / 共 60 页] 前端交互与视图组件模块 ====================
                  </div>
                  <div className="p-2.5 rounded bg-slate-50 dark:bg-slate-900 border border-slate-200 dark:border-
slate-80
                    <div className="text-[11px] text-slate-500">平均疏水性 (Kyte-Doolittle)</div>
                    <div className="text-base font-bold font-mono text-slate-800 dark:text-slate-200">
                      {wheelData.mean_hydrophobicity.toFixed(2)}
                    </div>
                  </div>
                </div>
                {/* Action Buttons */}
                <div className="flex items-center gap-2 pt-2 border-t border-slate-100 dark:border-slate-800">
                  <button
                    type="button"
                    onClick={() => {
                      addToCart({
                        sequence: wheelData.sequence,
                        name: 'Wheel-Analyzed',
                        p_amp: 0.9,
                        p_hemo: 0.5,
                        tsi: 0.7,
                      });
                    }}
                    className="btn-lab-secondary text-xs px-2.5 py-1.5 flex items-center gap-1"
                  >
                    <ShoppingCart size={13} /> 存入候选库
                  </button>
                  <button
                    type="button"
                    onClick={() => {
                      setActivePeptide(wheelData.sequence, 'Wheel-Target');
                      navigate('/evolution');
                    }}
                    className="btn-lab-secondary text-xs px-2.5 py-1.5 flex items-center gap-1 text-purple-600
dark:text-p
                  >
                    <Activity size={13} /> 送去突变
                  </button>
                  <button
                    type="button"
                    onClick={() => {
                      setActivePeptide(wheelData.sequence, 'Wheel-Target');
                      navigate('/report');
                    }}
                    className="btn-lab-secondary text-xs px-2.5 py-1.5 flex items-center gap-1 text-blue-600
dark:text-blu
                  >
                    <FileText size={13} /> 评估报告
                  </button>
                </div>
              </div>
            )}
            {/* Color Legend */}
            <div className="lab-panel p-4 flex flex-col gap-2 text-xs">
              <span className="lab-label">残基极性与电荷着色规范</span>
              <div className="grid grid-cols-2 gap-2 text-slate-600 dark:text-slate-400">
```

```
// ==================== [第 5 页 / 共 60 页] 前端交互与视图组件模块 ====================
0209                <div className="flex items-center gap-2">
0210                  <span className="w-3 h-3 rounded-full" style={{ backgroundColor: '#38bdf8' }}></span>
0211                  <span>碱性正电 (Basic: R, K, H)</span>
0212                </div>
0213                <div className="flex items-center gap-2">
0214                  <span className="w-3 h-3 rounded-full" style={{ backgroundColor: '#f43f5e' }}></span>
0215                  <span>酸性负电 (Acidic: D, E)</span>
0216                </div>
0217                <div className="flex items-center gap-2">
0218                  <span className="w-3 h-3 rounded-full" style={{ backgroundColor: '#f59e0b' }}></span>
0219                  <span>强疏水性 (Hydrophobic: L, I, V, F, W)</span>
0220                </div>
0221                <div className="flex items-center gap-2">
0222                  <span className="w-3 h-3 rounded-full" style={{ backgroundColor: '#fbbf24' }}></span>
0223                  <span>中度疏水 (Moderate: A, M, C)</span>
0224                </div>
0225                <div className="flex items-center gap-2">
0226                  <span className="w-3 h-3 rounded-full" style={{ backgroundColor: '#10b981' }}></span>
0227                  <span>极性不带电 (Polar: S, T, N, Q, Y, G, P)</span>
0228                </div>
0229              </div>
0230            </div>
0231          </div>
0232          {/* Right Section: SVG Helical Wheel Canvas */}
0233          <div className="lg:col-span-7 flex flex-col gap-4">
0234            <div className="lab-panel p-5 flex flex-col items-center justify-center min-h-[440px] relative">
0235              <div className="w-full flex items-center justify-between border-b border-slate-100 dark:border-
slate-800 pb-
0236                <span className="lab-label">Schiffer-Edmundson 100° 二维矢量投影图</span>
0237                <button
0238                  type="button"
0239                  onClick={handleExportPNG}
0240                  disabled={!wheelData}
0241                  className="btn-lab-secondary text-xs px-2.5 py-1 flex items-center gap-1.5 disabled:opacity-40"
0242                >
0243                  <Download size={13} /> 导出高清 PNG
0244                </button>
0245              </div>
0246              {loading ? (
0247                <div className="py-24 text-center text-slate-400 flex flex-col items-center justify-center gap-2">
0248                  <RefreshCw size={24} className="animate-spin text-amber-500" />
0249                  <span className="text-xs">计算残基空间角与两亲性力矩中...</span>
0250                </div>
0251              ) : !wheelData ? (
0252                <div className="py-24 text-center text-slate-400 text-xs">
0253                  请输入多肽序列后点击 “投影” 以生成螺旋轮。
0254                </div>
0255              ) : (
0256                <div className="w-full max-w-[420px] aspect-square flex items-center justify-center p-2">
0257                  <svg
0258                    ref={svgRef}
0259                    viewBox="0 0 320 320"
0260                    className="w-full h-full select-none"
```

```
// ==================== [第 6 页 / 共 60 页] 前端交互与视图组件模块 ====================
                  >
                    <defs>
                      <marker id="arrow" viewBox="0 0 10 10" refX="5" refY="5" markerWidth="6" markerHeight="6"
orient="au
                        <path d="M 0 0 L 10 5 L 0 10 z" fill="#f59e0b" />
                      </marker>
                    </defs>
                    {/* Concentric Guide Circles */}
                    <circle cx="160" cy="160" r="120" fill="none" stroke="#cbd5e1" strokeWidth="1"
strokeDasharray="3,3" c
                    <circle cx="160" cy="160" r="70" fill="none" stroke="#e2e8f0" strokeWidth="1"
strokeDasharray="2,2" cl
                    <circle cx="160" cy="160" r="3" fill="#94a3b8" />
                    {/* 18-Spoke Angular Rays */}
                    {Array.from({ length: 18 }).map((_, s) => {
                      const rayAngle = (s * 20 * Math.PI) / 180;
                      const rx = 160 + 120 * Math.sin(rayAngle);
                      const ry = 160 - 120 * Math.cos(rayAngle);
                      return (
                        <line
                          key={s}
                          x1="160" y1="160" x2={rx} y2={ry}
                          stroke="#f1f5f9" strokeWidth="0.7"
                          className="dark:stroke-slate-800/40"
                        />
                      );
                    })}
                    {/* Consecutive Backbone Connection Lines */}
                    {wheelData.residues.map((r, i) => {
                      if (i === 0) return null;
                      const prev = wheelData.residues[i - 1];
                      return (
                        <line
                          key={`line-${i}`}
                          x1={prev.cx}
                          y1={prev.cy}
                          x2={r.cx}
                          y2={r.cy}
                          stroke="#94a3b8"
                          strokeWidth="1"
                          opacity="0.5"
                        />
                      );
                    })}
                    {/* Eisenberg Hydrophobic Moment Vector Arrow */}
                    <line
                      x1={cx0}
                      y1={cy0}
                      x2={arrowX}
                      y2={arrowY}
                      stroke="#f59e0b"
                      strokeWidth="2.5"
                      markerEnd="url(#arrow)"
                    />
                    <text
```

```
// ==================== [第 7 页 / 共 60 页] 前端交互与视图组件模块 ====================
                      x={arrowX + 8}
                      y={arrowY + 4}
                      fill="#f59e0b"
                      fontSize="9"
                      fontWeight="bold"
                      fontFamily="monospace"
                    >
                      μH ({wheelData.hydrophobic_moment.toFixed(2)})
                    </text>
                    {/* Residue Circles */}
                    {wheelData.residues.map((r) => (
                      <g key={`res-${r.idx}`} className="cursor-pointer">
                        <circle
                          cx={r.cx}
                          cy={r.cy}
                          r="11"
                          fill={r.color}
                          stroke="#ffffff"
                          strokeWidth="1.5"
                          className="shadow-md"
                        />
                        <text
                          x={r.cx}
                          y={r.cy + 3.5}
                          fill="#ffffff"
                          fontSize="9"
                          fontWeight="bold"
                          fontFamily="monospace"
                          textAnchor="middle"
                        >
                          {r.residue}
                        </text>
                        <text
                          x={r.cx + (r.cx > 160 ? 13 : -13)}
                          y={r.cy + 3}
                          fill="#64748b"
                          fontSize="7"
                          fontFamily="monospace"
                          textAnchor={r.cx > 160 ? 'start' : 'end'}
                        >
                          {r.idx + 1}
                        </text>
                      </g>
                    ))}
                  </svg>
                </div>
              )}
            </div>
          </div>
        </div>
      </div>
    );
```

// ==================== [第 8 页 / 共 60 页] 前端交互与视图组件模块 ====================

```
}
import { useState, useEffect, useRef } from 'react';
import { apiClient, errorMessage, submitBatchJob, getBatchJob, cancelBatchJob, type BatchJobResponse } from
'../api/clie
import { useAppStore } from '../store/useAppStore';
type Mode = 'screening' | 'evolution' | 'report';
type Residue = { pos: number; aa: string; hydrophobicity: number; charge: number };
type Score = { sequence: string; amp_probability: number; hemo_probability: number; ranking_score?: number;
therapeutic_
type Candidate = { mutation: string; mutated_seq: string; amp_prob: number; hemo_prob: number; tsi: number;
rationale: s
type Report = { standard_markdown_report: string; provenance: Record<string, unknown> };
const titles = { screening: '双任务研究预测', evolution: '探索性突变排序', report: '计算研究报告' };
function parseSequences(input: string): string[] {
  const lines = input.split(/\r?\n/).map(x => x.trim()).filter(Boolean);
  if (!lines.some(x => x.startsWith('>'))) return lines;
  const sequences: string[] = [];
  let current = '';
  for (const line of lines) {
    if (line.startsWith('>')) {
      if (current) sequences.push(current);
      current = '';
    } else {
      current += line;
    }
  }
  if (current) sequences.push(current);
  return sequences;
}
const statusConfig: Record<string, { label: string; badgeClass: string }> = {
  queued: { label: '队列中', badgeClass: 'bg-amber-100 text-amber-800 dark:bg-amber-900/40 dark:text-amber-300
border-amber
  running: { label: '执行中', badgeClass: 'bg-blue-100 text-blue-800 dark:bg-blue-900/40 dark:text-blue-300
border-blue-300
  completed: { label: '已完成', badgeClass: 'bg-emerald-100 text-emerald-800 dark:bg-emerald-900/40 dark:text-
emerald-300 b
  cancelled: { label: '已取消', badgeClass: 'bg-slate-100 text-slate-800 dark:bg-slate-800 dark:text-slate-300
border-slate
  failed: { label: '失败', badgeClass: 'bg-rose-100 text-rose-800 dark:bg-rose-900/40 dark:text-rose-300 border-
rose-300 d
};
export default function ResearchStudio({ mode }: { mode: Mode }) {
  const { activeSequence, addToCart } = useAppStore();
  const [input, setInput] = useState(activeSequence);
  const [pending, setPending] = useState(false);
  const [error, setError] = useState('');
  const [scores, setScores] = useState<Score[]>([]);
  const [candidates, setCandidates] = useState<Candidate[] | null>(null);
  const [report, setReport] = useState<Report | null>(null);
  const [provenance, setProvenance] = useState<Record<string, unknown> | null>(null);
  // Batch job async queue states
  const [asyncQueueMode, setAsyncQueueMode] = useState(false);
  const [activeJobId, setActiveJobId] = useState<string | null>(null);
  const [jobProgress, setJobProgress] = useState<{
    completed: number;
    total: number;
    failed_count: number;
    status: BatchJobResponse['status'] | string;
  } | null>(null);
  const [jobErrors, setJobErrors] = useState<Array<{ index: number; sequence: string; error: string }>>([]);
```

```
// ==================== [第 9 页 / 共 60 页] 前端交互与视图组件模块 ====================
  const pollIntervalRef = useRef<ReturnType<typeof setInterval> | null>(null);
  const stopPolling = () => {
    if (pollIntervalRef.current) {
      clearInterval(pollIntervalRef.current);
      pollIntervalRef.current = null;
    }
  };
  useEffect(() => {
    return () => {
      stopPolling();
    };
  }, []);
  async function handleCancelJob() {
    if (!activeJobId) return;
    try {
      await cancelBatchJob(activeJobId);
      stopPolling();
      setJobProgress(prev => (prev ? { ...prev, status: 'cancelled' } : null));
      setActiveJobId(null);
      setPending(false);
      setError('作业已被用户取消。');
    } catch (err) {
      setError(errorMessage(err));
    }
  }
  async function run() {
    stopPolling();
    setError('');
    setScores([]);
    setCandidates(null);
    setReport(null);
    setProvenance(null);
    setActiveJobId(null);
    setJobProgress(null);
    setJobErrors([]);
    const sequences = parseSequences(input);
    if (!sequences.length || (mode !== 'screening' && sequences.length !== 1)) {
      setError(mode === 'screening' ? '请输入至少一条序列。' : '此功能每次接收一条序列。');
      return;
    }
    setPending(true);
    try {
      if (mode === 'screening') {
        const useAsync = sequences.length > 15 || asyncQueueMode;
        if (useAsync) {
          const job = await submitBatchJob(sequences);
          const jobId = job.job_id;
          setActiveJobId(jobId);
          setJobProgress({
            completed: 0,
            total: job.total || sequences.length,
            failed_count: 0,
```

```
// ==================== [第 10 页 / 共 60 页] 前端交互与视图组件模块 ====================
            status: 'queued',
          });
          // Poll getBatchJob(jobId) every 800ms
          pollIntervalRef.current = setInterval(async () => {
            try {
              const res = await getBatchJob(jobId);
              setJobProgress({
                completed: res.completed,
                total: res.total,
                failed_count: res.failed_count,
                status: res.status,
              });
              if (res.status === 'completed') {
                stopPolling();
                setActiveJobId(null);
                setPending(false);
                setScores(res.results as Score[]);
                if (res.results?.[0]?.provenance) {
                  setProvenance(res.results[0].provenance);
                }
                if (res.errors && res.errors.length > 0) {
                  setJobErrors(res.errors);
                }
              } else if (res.status === 'failed') {
                stopPolling();
                setActiveJobId(null);
                setPending(false);
                if (res.errors && res.errors.length > 0) {
                  setJobErrors(res.errors);
                  setError(`作业执行失败: ${res.errors[0]?.error || '未知错误'}`);
                } else {
                  setError('作业执行失败。');
                }
              } else if (res.status === 'cancelled') {
                stopPolling();
                setActiveJobId(null);
                setPending(false);
                setError('作业已取消。');
              }
            } catch (pollErr) {
              console.error('Polling error:', pollErr);
            }
          }, 800);
          return;
        } else {
          const { data } = await apiClient.post<Score[]>('/api/predict_batch', { sequences });
          setScores(data);
          setProvenance(data[0]?.provenance ?? null);
          setPending(false);
        }
      } else if (mode === 'evolution') {
        const { data } = await apiClient.post('/api/ai_optimize', { sequence: sequences[0] });
```

```
// ==================== [第 11 页 / 共 60 页] 前端交互与视图组件模块 ====================
          setCandidates(data.solutions);
          setProvenance(data.provenance);
          setPending(false);
        } else {
          const { data } = await apiClient.post<Report>('/api/ai_report', { sequence: sequences[0] });
          setReport(data);
          setProvenance(data.provenance);
          setPending(false);
        }
      } catch (err) {
        stopPolling();
        setActiveJobId(null);
        setPending(false);
        setError(errorMessage(err));
      }
    }
    function exportResult() {
      const content = report?.standard_markdown_report ?? JSON.stringify({ scores, candidates, provenance }, null,
2);
      const blob = new Blob([content], { type: report ? 'text/markdown;charset=utf-8' : 'application/json' });
      const url = URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download = 'safeamp-research.' + (report ? 'md' : 'json');
      link.click();
      URL.revokeObjectURL(url);
    }
    function exportCSV() {
      if (!scores.length) return;
      const header = 'Sequence,Length,P_AMP,P_Hemo,RankingScore\n';
      const rows = scores.map(s => {
        const score = s.ranking_score ?? s.therapeutic_safety_index ?? (s.amp_probability * (1 -
s.hemo_probability));
        return `"${s.sequence}",${s.sequence.length},${s.amp_probability.toFixed(4)},$
{s.hemo_probability.toFixed(4)},${sc
      }).join('\n');
      const blob = new Blob([header + rows], { type: 'text/csv;charset=utf-8;' });
      const url = URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download = `safeamp-screening-results-${Date.now()}.csv`;
      link.click();
      URL.revokeObjectURL(url);
    }
    return (
      <div className="flex flex-col gap-5">
        <h2 className="text-xl font-bold">{titles[mode]}</h2>
        <div className="lab-panel p-4 text-sm space-y-2">
          <p>适用范围：10–50 个标准氨基酸、未修饰序列。抗菌端点为大肠杆菌 MIC；溶血端点为人红细胞 HC50。</p>
          <p>概率对应预定义实验标签，不是杀菌率或临床安全性。排序分数 P(amp) × (1 − P(hemo)) 不代表治疗窗口。</p>
          {mode === 'evolution' && <p>候选由同一预测器产生并评分，仅供探索；分数提高不是独立验证，也不能证明实测降毒或活性
保留。</p>}
          {mode === 'report' && <p>报告在本地生成，记录运行来源与局限；尚无已核实的本项目发表记录可供引用。</p>}
        </div>
        <div className="lab-panel p-4 space-y-3">
          <label htmlFor={'research-' + mode} className="lab-label">
```

```
// ==================== [第 12 页 / 共 60 页] 前端交互与视图组件模块 ====================
0573            {mode === 'screening' ? '序列 / 多行 FASTA（最多 128 条）' : '单条序列'}
0574          </label>
0575          <textarea
0576            id={'research-' + mode}
0577            rows={mode === 'screening' ? 6 : 3}
0578            className="lab-input w-full font-mono text-sm"
0579            value={input}
0580            onChange={e => setInput(e.target.value)}
0581          />
0582          {mode === 'screening' && (
0583            <div className="flex items-center justify-between pt-1">
0584              <label className="inline-flex items-center gap-2 text-xs text-slate-600 dark:text-slate-400 cursor-
pointer s
0585                <input
0586                  type="checkbox"
0587                  checked={asyncQueueMode}
0588                  onChange={e => setAsyncQueueMode(e.target.checked)}
0589                  className="rounded border-slate-300 text-blue-600 focus:ring-blue-500"
0590                />
0591                <span>异步作业队列模式（&gt;15 条序列自动启用）</span>
0592              </label>
0593              <span className="text-xs text-slate-400 font-mono">
0594                已输入 {parseSequences(input).length} 条序列
0595              </span>
0596            </div>
0597          )}
0598          <div className="flex items-center gap-3">
0599            <button
0600              className="btn-lab-primary px-4 py-2 disabled:opacity-50"
0601              disabled={pending}
0602              onClick={run}
0603            >
0604              {pending ? '处理中…' : '运行' + titles[mode]}
0605            </button>
0606          </div>
0607          {error && <p role="alert" className="text-rose-600 dark:text-rose-300 break-words">{error}</p>}
0608        </div>
0609        {/* Laboratory-style Batch Progress Card */}
0610        {jobProgress && (
0611          <div className="lab-panel p-4 space-y-3 border-blue-200 dark:border-blue-900 bg-blue-50/20 dark:bg-
blue-950/10">
0612            <div className="flex items-center justify-between">
0613              <div className="flex items-center gap-2">
0614                <span className="text-sm font-semibold text-slate-800 dark:text-slate-200">
0615                  批量作业监视器 {activeJobId ? `[${activeJobId}]` : ''}
0616                </span>
0617                <span className={`px-2.5 py-0.5 text-xs font-semibold rounded-full border $
{statusConfig[jobProgress.statu
0618                  {statusConfig[jobProgress.status]?.label ?? jobProgress.status}
0619                </span>
0620              </div>
0621              {activeJobId && (jobProgress.status === 'queued' || jobProgress.status === 'running') && (
0622                <button
0623                  type="button"
0624                  className="btn-lab-secondary text-xs px-3 py-1 text-rose-600 dark:text-rose-400 hover:bg-rose-50
dark:ho
```

```
// ==================== [第 13 页 / 共 60 页] 前端交互与视图组件模块 ====================
                  onClick={handleCancelJob}
                >
                  取消作业
                </button>
              )}
            </div>
            <div className="space-y-1.5">
              <div className="flex justify-between text-xs font-mono text-slate-600 dark:text-slate-400">
                <span>已完成 {jobProgress.completed} / {jobProgress.total} (失败: {jobProgress.failed_count})</span>
                <span>{jobProgress.total > 0 ? Math.round((jobProgress.completed / jobProgress.total) * 100) :
0}%</span>
              </div>
              <div className="w-full bg-slate-200 dark:bg-slate-700 rounded-full h-2.5 overflow-hidden">
                <div
                  className="bg-blue-600 dark:bg-blue-500 h-2.5 rounded-full transition-all duration-300 ease-out"
                  style={{ width: `${jobProgress.total > 0 ? (jobProgress.completed / jobProgress.total) * 100 :
0}%` }}
                />
              </div>
            </div>
          </div>
        )}
        {/* Batch Job Error Alert Box */}
        {jobErrors.length > 0 && (
          <div role="alert" className="lab-panel p-4 border-rose-300 dark:border-rose-800 bg-rose-50/50 dark:bg-
rose-950/2
            <div className="font-semibold text-sm flex items-center gap-2">
              <span>⚠ 存在未通过计算的异常序列（共 {jobErrors.length} 条）：</span>
            </div>
            <ul className="list-disc list-inside text-xs space-y-1 font-mono max-h-48 overflow-y-auto">
              {jobErrors.map((err, idx) => (
                <li key={idx}>
                  {err.index >= 0 ? `序号 #${err.index + 1}` : '系统'}
                  {err.sequence ? ` [${err.sequence}...]` : ''}: {err.error}
                </li>
              ))}
            </ul>
          </div>
        )}
        {/* Scores Result Header & Export Bar */}
        {scores.length > 0 && (
          <div className="flex flex-wrap items-center justify-between gap-3 p-3 lab-panel bg-slate-50/50 dark:bg-
slate-800
            <div className="text-sm font-semibold text-slate-700 dark:text-slate-300">
              筛选结果列表（已完成 {scores.length} 条序列预测）
            </div>
            <div className="flex items-center gap-2">
              <button type="button" className="btn-lab-secondary text-xs px-3 py-1.5" onClick={exportCSV}>
                导出 CSV 表格
              </button>
              <button type="button" className="btn-lab-secondary text-xs px-3 py-1.5" onClick={exportResult}>
                导出 JSON 记录
              </button>
            </div>
          </div>
        )}
```

```
// ==================== [第 14 页 / 共 60 页] 前端交互与视图组件模块 ====================
        {/* Result Cards */}
        {scores.map((s, i) => {
          const score = s.ranking_score ?? s.therapeutic_safety_index ?? (s.amp_probability * (1 -
s.hemo_probability));
          return (
            <div className="lab-panel p-4 space-y-3" key={i}>
              <p className="font-mono break-all">{s.sequence}</p>
              <p>抗菌标签概率 {s.amp_probability.toFixed(4)} · 溶血标签概率 {s.hemo_probability.toFixed(4)} · 排序分数
{score.toFixed(4)}<
              <p className="text-sm text-slate-500">残基理化属性提示（固定规则，不是梯度归因或机制解释）</p>
              <div className="flex flex-wrap gap-1">
                {s.residues.map(r => (
                  <span
                    key={r.pos}
                    title={'位置 ' + r.pos + '; KD 疏水性 ' + r.hydrophobicity + '; 电荷估计 ' + r.charge}
                    className={'px-2 py-1 rounded font-mono ' + (r.hydrophobicity > 0 ? 'bg-amber-100 text-
amber-950 dark:
                  >
                    {r.aa}<sub>{r.pos}</sub>
                  </span>
                ))}
              </div>
              <button
                className="btn-lab-secondary px-3 py-1"
                onClick={() => addToCart({
                  sequence: s.sequence,
                  name: 'Research candidate',
                  p_amp: s.amp_probability,
                  p_hemo: s.hemo_probability,
                  tsi: score,
                  run_id: String(s.provenance?.run_id ?? 'research-run'),
                })}
              >
                存入候选库
              </button>
            </div>
          );
        })}
        {candidates !== null && (
          <div className="lab-panel p-4 space-y-3">
            {!candidates.length && <p>未找到满足当前计算筛选条件的候选；这不表明原序列安全或最优。</p>}
            {candidates.map(c => (
              <div key={c.mutation} className="border-b border-slate-200 dark:border-slate-700 pb-3">
                <strong>{c.mutation}</strong>
                <p className="font-mono break-all">{c.mutated_seq}</p>
                <p>抗菌标签概率 {c.amp_prob.toFixed(4)} · 溶血标签概率 {c.hemo_prob.toFixed(4)} · 排序分数
{c.tsi.toFixed(4)}</p>
                <p>{c.rationale}</p>
              </div>
            ))}
          </div>
        )}
        {report && (
          <pre className="lab-panel p-4 whitespace-pre-wrap break-words text-sm">
            {report.standard_markdown_report}
          </pre>
```

```
// ==================== [第 15 页 / 共 60 页] 前端交互与视图组件模块 ====================
        )}
        {provenance && (
          <div className="lab-panel p-4 space-y-3">
            <h3>运行来源</h3>
            <pre className="text-xs whitespace-pre-wrap break-words">{JSON.stringify(provenance, null, 2)}</pre>
            <div className="flex flex-wrap gap-2">
              <button className="btn-lab-secondary px-3 py-2" onClick={exportResult}>
                导出研究记录（JSON）
              </button>
              {scores.length > 0 && (
                <button className="btn-lab-secondary px-3 py-2" onClick={exportCSV}>
                  导出筛选结果（CSV）
                </button>
              )}
            </div>
          </div>
        )}
      </div>
    );
}
import React, { useState } from 'react';
import { Settings, X, Key, Sliders, ShieldAlert, Check, RotateCcw, Eye, EyeOff } from 'lucide-react';
import { useAppStore } from '../store/useAppStore';
interface Props {
  isOpen: boolean;
  onClose: () => void;
}
const PROVIDER_PRESETS: Record<string, { apiBase: string; modelName: string; label: string }> = {
  deepseek: {
    label: 'DeepSeek（默认）',
    apiBase: 'https://api.deepseek.com/v1',
    modelName: 'deepseek-chat',
  },
  openai: {
    label: 'OpenAI',
    apiBase: 'https://api.openai.com/v1',
    modelName: 'gpt-4o',
  },
  anthropic: {
    label: 'Anthropic Claude',
    apiBase: 'https://api.anthropic.com/v1',
    modelName: 'claude-3-5-sonnet-20241022',
  },
  ollama: {
    label: 'Ollama（本地私有部署）',
    apiBase: 'http://localhost:11434/v1',
    modelName: 'llama3:latest',
  },
  custom: {
    label: '自定义提供商（Custom）',
    apiBase: '',
    modelName: '',
```

```
// ==================== [第 16 页 / 共 60 页] 前端交互与视图组件模块 ====================
  },
};
export function SettingsModal({ isOpen, onClose }: Props) {
  const {
    llmProvider,
    apiKey,
    apiBase,
    modelName,
    temperature,
    setLLMConfig,
  } = useAppStore();
  const [provider, setProvider] = useState(llmProvider || 'deepseek');
  const [key, setKey] = useState(apiKey || '');
  const [base, setBase] = useState(apiBase || 'https://api.deepseek.com/v1');
  const [model, setModel] = useState(modelName || 'deepseek-chat');
  const [temp, setTemp] = useState(temperature ?? 0.0);
  const [showKey, setShowKey] = useState(false);
  const [savedSuccess, setSavedSuccess] = useState(false);
  if (!isOpen) return null;
  const handleProviderChange = (newProvider: string) => {
    setProvider(newProvider);
    const preset = PROVIDER_PRESETS[newProvider];
    if (preset && newProvider !== 'custom') {
      setBase(preset.apiBase);
      setModel(preset.modelName);
    }
  };
  const handleSave = () => {
    setLLMConfig({
      llmProvider: provider,
      apiKey: key.trim(),
      apiBase: base.trim(),
      modelName: model.trim(),
      temperature: temp,
    });
    setSavedSuccess(true);
    setTimeout(() => {
      setSavedSuccess(false);
      onClose();
    }, 600);
  };
  const handleReset = () => {
    const defaultPreset = PROVIDER_PRESETS['deepseek'];
    setProvider('deepseek');
    setKey('');
    setBase(defaultPreset.apiBase);
    setModel(defaultPreset.modelName);
    setTemp(0.0);
    setLLMConfig({
      llmProvider: 'deepseek',
      apiKey: '',
      apiBase: defaultPreset.apiBase,
```

```
// ==================== [第 17 页 / 共 60 页] 前端交互与视图组件模块 ====================
0833        modelName: defaultPreset.modelName,
0834        temperature: 0.0,
0835      });
0836    };
0837    return (
0838      <div className="fixed inset-0 z-[100] flex items-center justify-center bg-slate-900/60 backdrop-blur-xs p-4"
role="d
0839        <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-xl
shadow-2xl w-f
0840          {/* Header */}
0841          <div className="px-6 py-4 border-b border-slate-200 dark:border-slate-800 flex items-center justify-
between bg-s
0842            <div className="flex items-center gap-2.5 text-slate-800 dark:text-slate-100 font-semibold text-base">
0843              <Settings size={20} className="text-blue-600 dark:text-blue-400" />
0844              <span>配置中心 · 可拔插模型设置</span>
0845            </div>
0846            <button
0847              onClick={onClose}
0848              className="text-slate-400 hover:text-slate-600 dark:hover:text-slate-200 p-1 rounded-md transition-
colors"
0849            >
0850              <X size={18} />
0851            </button>
0852          </div>
0853          {/* Content */}
0854          <div className="p-6 overflow-y-auto flex-1 space-y-4">
0855            {/* Research Governance Banner */}
0856            <div className="p-3.5 rounded-lg border border-amber-200 dark:border-amber-900/60 bg-amber-50/70
dark:bg-amber
0857              <div className="flex items-center gap-2 font-semibold">
0858                <ShieldAlert size={16} className="text-amber-600 dark:text-amber-400 shrink-0" />
0859                <span>科研安全与确定性规范 (Research Provenance & Isolation)</span>
0860              </div>
0861              <p className="leading-relaxed">
0862                SafeAMP-Dual 报告工坊优先采用本地确定性算法 (Evidence-Bounded Local Engine) 以确保科研可重复性与序列防泄
露。设置外部大模型仅用于辅助报告文本生成，生物物理打分与突变筛选严
0863              </p>
0864            </div>
0865            {/* Provider Selection */}
0866            <div className="space-y-1.5">
0867              <label className="lab-label">大模型服务提供商 (LLM Provider)</label>
0868              <select
0869                value={provider}
0870                onChange={(e) => handleProviderChange(e.target.value)}
0871                className="lab-input text-xs"
0872              >
0873                {Object.entries(PROVIDER_PRESETS).map(([keyVal, preset]) => (
0874                  <option key={keyVal} value={keyVal}>
0875                    {preset.label}
0876                  </option>
0877                ))}
0878              </select>
0879            </div>
0880            {/* Model Name */}
0881            <div className="space-y-1.5">
0882              <label className="lab-label">模型标识 (Model Identifier)</label>
0883              <input
0884                type="text"
```

```
// ==================== [第 18 页 / 共 60 页] 前端交互与视图组件模块 ====================
                value={model}
                onChange={(e) => setModel(e.target.value)}
                placeholder="例如 deepseek-chat 或 gpt-4o"
                className="lab-input font-mono text-xs"
              />
            </div>
            {/* API Base URL */}
            <div className="space-y-1.5">
              <label className="lab-label">接口服务地址 (API Base URL)</label>
              <input
                type="text"
                value={base}
                onChange={(e) => setBase(e.target.value)}
                placeholder="https://api.deepseek.com/v1"
                className="lab-input font-mono text-xs"
              />
            </div>
            {/* API Key */}
            <div className="space-y-1.5">
              <div className="flex items-center justify-between">
                <label className="lab-label flex items-center gap-1">
                  <Key size={12} />
                  <span>API 访问凭证 (API Key)</span>
                </label>
                <button
                  type="button"
                  onClick={() => setShowKey(!showKey)}
                  className="text-[11px] text-slate-500 hover:text-slate-700 dark:hover:text-slate-300 flex items-
center g
                >
                  {showKey ? <EyeOff size={12} /> : <Eye size={12} />}
                  <span>{showKey ? '隐藏' : '显示'}</span>
                </button>
              </div>
              <input
                type={showKey ? 'text' : 'password'}
                value={key}
                onChange={(e) => setKey(e.target.value)}
                placeholder="sk-..."
                className="lab-input font-mono text-xs"
              />
              <p className="text-[11px] text-slate-400">
                凭证仅保存在浏览器本地会话与客户端缓存中，不上传至公共网络。
              </p>
            </div>
            {/* Temperature Slider */}
            <div className="space-y-2 pt-2 border-t border-slate-100 dark:border-slate-800">
              <div className="flex items-center justify-between">
                <label className="lab-label flex items-center gap-1 mb-0">
                  <Sliders size={12} />
                  <span>采样温度 (Temperature): {temp.toFixed(1)}</span>
                </label>
                <span className="text-[11px] font-mono px-2 py-0.5 rounded bg-slate-100 dark:bg-slate-800 text-
slate-600 d
```

```
// ==================== [第 19 页 / 共 60 页] 前端交互与视图组件模块 ====================
                  {temp === 0.0 ? '0.0 (确定性输出)' : temp < 0.5 ? '低发散度' : '高探索度'}
                </span>
              </div>
              <input
                type="range"
                min="0.0"
                max="1.0"
                step="0.1"
                value={temp}
                onChange={(e) => setTemp(parseFloat(e.target.value))}
                className="w-full accent-blue-600 cursor-pointer"
              />
              <div className="flex justify-between text-[10px] text-slate-400 font-mono">
                <span>0.0 (严谨科研报告)</span>
                <span>0.5 (平衡)</span>
                <span>1.0 (高度多样化)</span>
              </div>
            </div>
          </div>
          {/* Footer */}
          <div className="px-6 py-4 border-t border-slate-200 dark:border-slate-800 bg-slate-50/50 dark:bg-
slate-800/50 fl
            <button
              type="button"
              onClick={handleReset}
              className="text-xs text-slate-500 hover:text-slate-700 dark:hover:text-slate-300 flex items-center
gap-1 hov
            >
              <RotateCcw size={13} />
              <span>恢复默认</span>
            </button>
            <div className="flex items-center gap-2">
              <button
                type="button"
                onClick={onClose}
                className="btn-lab-secondary text-xs px-3 py-1.5"
              >
                取消
              </button>
              <button
                type="button"
                onClick={handleSave}
                className="btn-lab-primary text-xs px-4 py-1.5 flex items-center gap-1.5"
              >
                {savedSuccess ? <Check size={14} /> : null}
                <span>{savedSuccess ? '已保存配置' : '保存设置'}</span>
              </button>
            </div>
          </div>
        </div>
      </div>
    );
}
import React from 'react';
```

```
// ==================== [第 20 页 / 共 60 页] 前端交互与视图组件模块 ====================
import { ShoppingCart, X, Trash2, Download, Activity, Zap, FileText } from 'lucide-react';
import { useNavigate } from 'react-router-dom';
import { useAppStore } from '../store/useAppStore';
interface Props {
  isOpen: boolean;
  onClose: () => void;
}
export const PeptideCartModal: React.FC<Props> = ({ isOpen, onClose }) => {
  const { cart, removeFromCart, clearCart, setActivePeptide } = useAppStore();
  const navigate = useNavigate();
  if (!isOpen) return null;
  const handleExportCSV = () => {
    if (cart.length === 0) return;
    const header = 'Sequence,Name,P_AMP,P_Hemo,ComputationalRankingScore,AddedAt\n';
    const rows = cart.map(c => `"${c.sequence}","${c.name}",${c.p_amp},${c.p_hemo},${c.tsi},"$
{c.addedAt}"`).join('\n');
    const blob = new Blob([header + rows], { type: 'text/csv;charset=utf-8;' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = `SafeAMP_Candidate_Library_${Date.now()}.csv`;
    link.click();
    URL.revokeObjectURL(url);
  };
  const handleSendToEvolution = (seq: string, name: string) => {
    setActivePeptide(seq, name);
    onClose();
    navigate('/evolution');
  };
  const handleSendToWheel = (seq: string, name: string) => {
    setActivePeptide(seq, name);
    onClose();
    navigate('/wheel');
  };
  const handleSendToReport = (seq: string, name: string) => {
    setActivePeptide(seq, name);
    onClose();
    navigate('/report');
  };
  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center bg-slate-900/60 backdrop-blur-xs p-4">
      <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-xl
shadow-2xl w-f
        {/* Header */}
        <div className="px-6 py-4 border-b border-slate-200 dark:border-slate-800 flex items-center justify-
between bg-s
          <div className="flex items-center gap-2.5 text-slate-800 dark:text-slate-100 font-semibold text-base">
            <ShoppingCart size={20} className="text-blue-600 dark:text-blue-400" />
            <span>候选多肽暂存库 (Candidate Peptide Library)</span>
            <span className="text-xs bg-blue-100 text-blue-800 dark:bg-blue-900/60 dark:text-blue-300 px-2 py-0.5
rounde
              {cart.length} 序列
            </span>
          </div>
          <button onClick={onClose} className="text-slate-400 hover:text-slate-600 dark:hover:text-slate-200 p-1
rounded
            <X size={18} />
```

```
// ==================== [第 21 页 / 共 60 页] 前端交互与视图组件模块 ====================
            </button>
          </div>
          {/* List Content */}
          <div className="p-6 overflow-y-auto flex-1 divide-y divide-slate-100 dark:divide-slate-800">
            {cart.length === 0 ? (
              <div className="py-12 text-center text-slate-400 dark:text-slate-600 flex flex-col items-center
justify-cent
                <ShoppingCart size={40} className="stroke-[1.5]" />
                <p>暂存库为空。在筛选台或突变实验室中点击“存入候选库”即可暂存分子。</p>
              </div>
            ) : (
              cart.map(item => (
                <div key={item.id} className="py-3.5 flex flex-col sm:flex-row items-start sm:items-center justify-
between
                  <div className="min-w-0 flex-1">
                    <div className="flex items-center gap-2 mb-1">
                      <span className="text-xs font-semibold text-slate-800 dark:text-slate-200">{item.name}</span>
                      <span className="text-[10px] text-slate-400 font-mono">({item.sequence.length} AA)</span>
                      <span className={`text-[10px] px-1.5 py-0.2 rounded font-mono font-medium ${
                        item.tsi >= 0.5
                          ? 'bg-emerald-50 text-emerald-700 dark:bg-emerald-950/40 dark:text-emerald-400 border
border-eme
                          : 'bg-rose-50 text-rose-700 dark:bg-rose-950/40 dark:text-rose-400 border border-rose-200
dark:b
                      }`}>
                        {item.run_id ? '排序分数: ' + item.tsi.toFixed(3) : '未关联验证运行，需重新预测'}
                      </span>
                    </div>
                    <div className="font-mono text-xs text-slate-600 dark:text-slate-300 break-all select-all">
                      {item.sequence}
                    </div>
                    <div className="flex items-center gap-4 text-[11px] text-slate-500 font-mono mt-1">
                      <span>P_AMP: {item.run_id ? item.p_amp.toFixed(3) : '未计算'}</span>
                      <span>P_Hemo: {item.run_id ? item.p_hemo.toFixed(3) : '未计算'}</span>
                      <span>添加时间: {item.addedAt}</span>
                    </div>
                  </div>
                  <div className="flex items-center gap-1.5 shrink-0 self-end sm:self-center">
                    <button
                      onClick={() => handleSendToEvolution(item.sequence, item.name)}
                      title="送入突变实验室"
                      className="p-1.5 text-xs text-purple-600 dark:text-purple-400 hover:bg-purple-50
dark:hover:bg-purpl
                    >
                      <Activity size={13} /> 突变
                    </button>
                    <button
                      onClick={() => handleSendToWheel(item.sequence, item.name)}
                      title="绘制螺旋轮构象"
                      className="p-1.5 text-xs text-amber-600 dark:text-amber-400 hover:bg-amber-50 dark:hover:bg-
amber-95
                    >
                      <Zap size={13} /> 螺旋
                    </button>
                    <button
                      onClick={() => handleSendToReport(item.sequence, item.name)}
                      title="生成计算研究报告"
                      className="p-1.5 text-xs text-blue-600 dark:text-blue-400 hover:bg-blue-50 dark:hover:bg-
blue-950/50
```

```
// ==================== [第 22 页 / 共 60 页] 前端交互与视图组件模块 ====================
                    >
                      <FileText size={13} /> 报告
                    </button>
                    <button
                      onClick={() => removeFromCart(item.id)}
                      title="删除"
                      className="p-1.5 text-slate-400 hover:text-rose-600 dark:hover:text-rose-400 rounded
transition-colo
                    >
                      <Trash2 size={14} />
                    </button>
                  </div>
                </div>
              ))
            )}
          </div>
          {/* Footer */}
          <div className="px-6 py-4 border-t border-slate-200 dark:border-slate-800 bg-slate-50/50 dark:bg-
slate-800/50 fl
            <button
              type="button"
              onClick={clearCart}
              disabled={cart.length === 0}
              className="text-xs text-rose-600 dark:text-rose-400 hover:underline disabled:opacity-40"
            >
              清空库
            </button>
            <div className="flex items-center gap-3">
              <button
                type="button"
                onClick={handleExportCSV}
                disabled={cart.length === 0}
                className="inline-flex items-center gap-1.5 px-3 py-1.5 text-xs font-medium text-slate-700
dark:text-slate
              >
                <Download size={13} /> 导出为 CSV
              </button>
              <button
                type="button"
                onClick={onClose}
                className="px-4 py-1.5 text-xs font-semibold text-white bg-blue-600 hover:bg-blue-700 rounded-lg
shadow-xs
              >
                完成
              </button>
            </div>
          </div>
        </div>
      </div>
    );
  };
  import React, { Component, type ErrorInfo, type ReactNode } from 'react';
  import { AlertTriangle, RotateCcw } from 'lucide-react';
  interface Props {
    children: ReactNode;
    fallbackTitle?: string;
```

```
// ==================== [第 23 页 / 共 60 页] 前端交互与视图组件模块 ====================
}
interface State {
  hasError: boolean;
  error: Error | null;
}
export class ErrorBoundary extends Component<Props, State> {
  public state: State = {
    hasError: false,
    error: null,
  };
  public static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }
  public componentDidCatch(error: Error, errorInfo: ErrorInfo) {
    console.error('Uncaught error in UI component:', error, errorInfo);
  }
  public handleReset = () => {
    this.setState({ hasError: false, error: null });
  };
  public render() {
    if (this.state.hasError) {
      return (
        <div className="p-6 my-4 rounded-xl border border-rose-200 dark:border-rose-900 bg-rose-50/70 dark:bg-
rose-950/3
          <div className="flex items-center gap-2">
            <AlertTriangle className="text-rose-600 dark:text-rose-400" size={20} />
            <h3 className="font-semibold text-base">
              {this.props.fallbackTitle || '页面组件运行异常'}
            </h3>
          </div>
          <p className="text-xs text-rose-700 dark:text-rose-300 font-mono break-all">
            {this.state.error?.message || '发生了未捕获的渲染错误'}
          </p>
          <div className="flex items-center gap-3 pt-2">
            <button
              type="button"
              onClick={this.handleReset}
              className="inline-flex items-center gap-1.5 px-3 py-1.5 text-xs font-semibold text-white bg-
rose-600 hover
            >
              <RotateCcw size={14} />
              <span>重试该组件</span>
            </button>
            <button
              type="button"
              onClick={() => window.location.reload()}
              className="px-3 py-1.5 text-xs font-semibold text-rose-700 dark:text-rose-300 bg-white dark:bg-
slate-800 b
            >
              刷新整个页面
            </button>
          </div>
        </div>
      );
    }
```

```
// ==================== [第 24 页 / 共 60 页] 前端交互与视图组件模块 ====================
      return this.props.children;
    }
  }
  import { create } from 'zustand';
  import { persist } from 'zustand/middleware';
  export interface CartPeptide {
    id: string;
    sequence: string;
    name: string;
    p_amp: number;
    p_hemo: number;
    tsi: number;
    is_short?: boolean;
    addedAt: string;
    run_id?: string;
  }
  interface AppState {
    // 可拔插大模型配置 (Pluggable Model Config)
    llmProvider: string;
    apiKey: string;
    apiBase: string;
    modelName: string;
    temperature: number;
    setLLMConfig: (config: Partial<Pick<AppState, 'llmProvider' | 'apiKey' | 'apiBase' | 'modelName' |
'temperature'>>) =>
    // 跨模块流通分子 (Active Peptide across Studios)
    activeSequence: string;
    activePeptideName: string;
    setActivePeptide: (seq: string, name?: string) => void;
    // 候选分子库 (Peptide Cart)
    cart: CartPeptide[];
    addToCart: (peptide: Omit<CartPeptide, 'id' | 'addedAt'>) => boolean;
    removeFromCart: (id: string) => void;
    clearCart: () => void;
  }
  export const useAppStore = create<AppState>()(
    persist(
      (set, get) => ({
        llmProvider: 'deepseek',
        apiKey: '',
        apiBase: 'https://api.deepseek.com/v1',
        modelName: 'deepseek-chat',
        temperature: 0.0,
        setLLMConfig: (config) => set((state) => ({ ...state, ...config })),
        activeSequence: 'GIGAVLKVLTTGLPALISWIKRKRQQ',
        activePeptideName: 'Melittin (蜂毒素)',
        setActivePeptide: (seq, name) => set({
          activeSequence: seq.trim().toUpperCase(),
          activePeptideName: name || 'Custom Peptide'
        }),
        cart: [],
        addToCart: (peptide) => {
          const { cart } = get();
```

```
// ==================== [第 25 页 / 共 60 页] 前端交互与视图组件模块 ====================
        const exists = cart.some(p => p.sequence === peptide.sequence);
        if (exists) return false;
        const newEntry: CartPeptide = {
          ...peptide,
          id: `${Date.now()}-${Math.random().toString(36).substr(2, 4)}`,
          addedAt: new Date().toLocaleTimeString(),
        };
        set({ cart: [newEntry, ...cart] });
        return true;
      },
      removeFromCart: (id) => set((state) => ({ cart: state.cart.filter(p => p.id !== id) })),
      clearCart: () => set({ cart: [] }),
    }),
    {
      name: 'safeamp-lab-workspace-v3',
      version: 4,
      partialize: (state) => ({
        activeSequence: state.activeSequence,
        activePeptideName: state.activePeptideName,
        cart: state.cart,
        llmProvider: state.llmProvider,
        apiKey: state.apiKey,
        apiBase: state.apiBase,
        modelName: state.modelName,
        temperature: state.temperature,
      }),
      migrate: (state: any) => ({
        activeSequence: state?.activeSequence ?? '',
        activePeptideName: state?.activePeptideName ?? '',
        cart: (Array.isArray(state?.cart) ? state.cart : []) as CartPeptide[],
        llmProvider: state?.llmProvider ?? 'deepseek',
        apiKey: state?.apiKey ?? '',
        apiBase: state?.apiBase ?? 'https://api.deepseek.com/v1',
        modelName: state?.modelName ?? 'deepseek-chat',
        temperature: state?.temperature ?? 0.0,
      }),
    }
  )
);
import axios from 'axios';
export const apiClient = axios.create({
  baseURL: '/',
  timeout: 30000,
  headers: {
    'Content-Type': 'application/json',
  },
});
export function errorMessage(error: unknown): string {
  if (!axios.isAxiosError(error)) return error instanceof Error ? error.message : '请求失败';
  if (error.code === 'ECONNABORTED' || error.message.includes('timeout')) {
    return '请求超时（超过 30 秒），请检查网络连接或后端服务器状态。';
  }
```

```
// ==================== [第 26 页 / 共 60 页] 前端交互与视图组件模块 ====================
    if (!error.response && error.request) {
      return '无法连接到后端服务，请确认本地后端已启动（端口 8000/8090）。';
    }
    const detail = error.response?.data?.detail;
    if (typeof detail === 'string') return detail;
    if (Array.isArray(detail)) return detail.map(item => item.msg ?? '输入不合法').join('; ');
    if (detail?.code === 'MODEL_NOT_VALIDATED') return '模型不可用: ' + detail.message;
    return detail?.message ?? error.message ?? '请求失败';
  }
  export interface BatchJobResponse {
    job_id: string;
    status: 'queued' | 'running' | 'completed' | 'failed' | 'cancelled';
    total: number;
    completed: number;
    failed_count: number;
    results: any[];
    errors: Array<{ index: number; sequence: string; error: string }>;
    created_at: number;
    updated_at: number;
  }
  export async function submitBatchJob(sequences: string[]): Promise<{ job_id: string; status: string; total:
number }> {
    const res = await apiClient.post('/api/batch_jobs', { sequences });
    return res.data;
  }
  export async function getBatchJob(jobId: string): Promise<BatchJobResponse> {
    const res = await apiClient.get(`/api/batch_jobs/${jobId}`);
    return res.data;
  }
  export async function cancelBatchJob(jobId: string): Promise<{ job_id: string; status: string }> {
    const res = await apiClient.delete(`/api/batch_jobs/${jobId}`);
    return res.data;
  }
  import React, { useState, useEffect } from 'react';
  import { BrowserRouter, Routes, Route, NavLink, Outlet, useLocation } from 'react-router-dom';
  import { QueryClient, QueryClientProvider, useQuery } from '@tanstack/react-query';
  import { apiClient } from './api/client';
  import { Activity, Beaker, Dna, FileBarChart, Settings, ShoppingCart, Zap, Sun, Moon } from 'lucide-react';
  import { useAppStore } from './store/useAppStore';
  import ScreeningStudio from './pages/ScreeningStudio';
  import EvolutionStudio from './pages/EvolutionStudio';
  import HelicalWheelStudio from './pages/HelicalWheelStudio';
  import ReportStudio from './pages/ReportStudio';
  import { SettingsModal } from './components/SettingsModal';
  import { PeptideCartModal } from './components/PeptideCartModal';
  import { ErrorBoundary } from './components/ErrorBoundary';
  const queryClient = new QueryClient();
  const NavTab = ({ to, icon: Icon, label, isActive }: { to: string, icon: any, label: string, isActive: boolean })
=> (
    <NavLink to={to}>
      <div className={`flex items-center gap-2 px-3 py-2 rounded-md text-xs font-semibold transition-all ${
        isActive
          ? 'bg-blue-600 text-white shadow-xs'
          : 'text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-slate-100 hover:bg-slate-100
dark:hov
```

```
// ==================== [第 27 页 / 共 60 页] 前端交互与视图组件模块 ====================
      }`}>
        <Icon size={15} />
        <span>{label}</span>
      </div>
    </NavLink>
  );
  const GlobalHeader = ({
    isDark,
    toggleDark,
    openSettings,
    openCart
  }: {
    isDark: boolean;
    toggleDark: () => void;
    openSettings: () => void;
    openCart: () => void;
  }) => {
    const { cart } = useAppStore();
    const status = useQuery({ queryKey: ['research-status'], queryFn: async () => (await
apiClient.get('/api/system_status
    const ready = status.data?.status === 'healthy';
    return (
      <header className="sticky top-0 z-40 bg-white/95 dark:bg-slate-900/95 backdrop-blur-xs border-b border-
slate-200 dar
        <div className="max-w-7xl mx-auto flex flex-wrap gap-2 items-center justify-between">
          {/* Brand */}
          <div className="flex items-center gap-3">
            <div className="w-8 h-8 rounded-lg bg-blue-600 flex items-center justify-center text-white shadow-xs">
              <Dna size={18} />
            </div>
            <div>
              <div className="flex items-center gap-2">
                <h1 className="text-sm font-bold tracking-tight text-slate-900 dark:text-slate-100">SafeAMP-Dual
Research<
                <span className="text-[10px] bg-slate-100 dark:bg-slate-800 text-slate-600 dark:text-slate-400
px-1.5 py-0
                  v4.0.0
                </span>
              </div>
              <p className="text-[10px] text-slate-400 dark:text-slate-500 font-mono tracking-wider uppercase">
                计算生物学与抗微生物肽工程实验工作台
              </p>
            </div>
          </div>
          {/* System Online Badges */}
          <div role="status" className={'text-xs px-2 py-1 rounded border ' + (ready ? 'text-emerald-700 border-
emerald-50
            {status.isPending ? '检查模型状态…' : ready ? '研究模型已通过运行校验' : '研究模型不可用'}
          </div>
          {/* Global Action Tools */}
          <div className="flex items-center gap-2">
            <button
              type="button"
              onClick={toggleDark}
              title="切换日间/夜间模式"
              className="p-1.5 text-slate-500 hover:text-slate-700 dark:hover:text-slate-300 rounded-md hover:bg-
slate-100
            >
```

```
// ==================== [第 28 页 / 共 60 页] 前端交互与视图组件模块 ====================
              {isDark ? <Sun size={16} /> : <Moon size={16} />}
            </button>
            <button
              type="button"
              onClick={openSettings}
              title="配置中心 (可拔插模型设置)"
              className="p-1.5 text-slate-500 hover:text-slate-700 dark:hover:text-slate-300 rounded-md hover:bg-
slate-100
            >
              <Settings size={16} />
              <span className="hidden sm:inline">配置中心</span>
            </button>
            <button
              type="button"
              onClick={openCart}
              title="查看暂存的候选多肽分子库"
              className="inline-flex items-center gap-1.5 px-2.5 py-1 text-xs font-semibold text-slate-700
dark:text-slate
            >
              <ShoppingCart size={14} />
              <span>分子库</span>
              <span className="bg-blue-600 text-white text-[10px] px-1.5 py-0.2 rounded-full font-mono">
                {cart.length}
              </span>
            </button>
          </div>
        </div>
      </header>
    );
  };
  const Layout = () => {
    const location = useLocation();
    const [isDark, setIsDark] = useState(false);
    const [settingsOpen, setSettingsOpen] = useState(false);
    const [cartOpen, setCartOpen] = useState(false);
    useEffect(() => {
      if (isDark) {
        document.documentElement.classList.add('dark');
      } else {
        document.documentElement.classList.remove('dark');
      }
    }, [isDark]);
    return (
      <div className="min-h-screen flex flex-col bg-slate-50 dark:bg-slate-950 text-slate-800 dark:text-slate-200">
        <GlobalHeader
          isDark={isDark}
          toggleDark={() => setIsDark(!isDark)}
          openSettings={() => setSettingsOpen(true)}
          openCart={() => setCartOpen(true)}
        />
        <SettingsModal isOpen={settingsOpen} onClose={() => setSettingsOpen(false)} />
        <PeptideCartModal isOpen={cartOpen} onClose={() => setCartOpen(false)} />
        {/* Main Studio Navigation Bar */}
        <div className="bg-white dark:bg-slate-900 border-b border-slate-200 dark:border-slate-800">
```

```
// ==================== [第 29 页 / 共 60 页] 前端交互与视图组件模块 ====================
          <div className="max-w-7xl mx-auto px-6 py-1.5 flex items-center gap-2 overflow-x-auto">
            <NavTab to="/" icon={Beaker} label="1. 高通量协同筛选" isActive={location.pathname === '/'} />
            <NavTab to="/evolution" icon={Activity} label="2. 探索性突变排序" isActive={location.pathname ===
'/evolution'} />
            <NavTab to="/wheel" icon={Zap} label="3. 螺旋轮与两亲投影" isActive={location.pathname === '/wheel'} />
            <NavTab to="/report" icon={FileBarChart} label="4. 科研报告与导出" isActive={location.pathname ===
'/report'} />
          </div>
        </div>
        {/* Main Workstation Canvas */}
        <main className="flex-1 max-w-7xl w-full mx-auto p-4 sm:p-6">
          <p className="mb-5 p-3 rounded border border-amber-300 dark:border-amber-800 bg-amber-50 dark:bg-
amber-950/40 te
          <ErrorBoundary fallbackTitle="科研工作台页面组件运行异常">
            <Outlet />
          </ErrorBoundary>
        </main>
        {/* Scientific Lab Footer */}
        <footer className="border-t border-slate-200 dark:border-slate-800 bg-white/50 dark:bg-slate-900/50 py-3
text-cent
          SafeAMP-Dual · Computational research prototype · No clinical validation
        </footer>
      </div>
    );
  };
  export default function App() {
    return (
      <QueryClientProvider client={queryClient}>
        <BrowserRouter>
          <Routes>
            <Route path="/" element={<Layout />}>
              <Route index element={<ScreeningStudio />} />
              <Route path="evolution" element={<EvolutionStudio />} />
              <Route path="wheel" element={<HelicalWheelStudio />} />
              <Route path="report" element={<ReportStudio />} />
            </Route>
          </Routes>
        </BrowserRouter>
      </QueryClientProvider>
    );
  }
  .counter {
    font-size: 16px;
    padding: 5px 10px;
    border-radius: 5px;
    color: var(--accent);
    background: var(--accent-bg);
    border: 2px solid transparent;
    transition: border-color 0.3s;
    margin-bottom: 24px;
    &:hover {
      border-color: var(--accent-border);
    }
    &:focus-visible {
      outline: 2px solid var(--accent);
      outline-offset: 2px;
```

```
// ==================== [第 30 页 / 共 60 页] 前端交互与视图组件模块 ====================
    }
  }
  .hero {
    position: relative;
    .base,
    .framework,
    .vite {
      inset-inline: 0;
      margin: 0 auto;
    }
    .base {
      width: 170px;
      position: relative;
      z-index: 0;
    }
    .framework,
    .vite {
      position: absolute;
    }
    .framework {
      z-index: 1;
      top: 34px;
      height: 28px;
      transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
        scale(1.4);
    }
    .vite {
      z-index: 0;
      top: 107px;
      height: 26px;
      width: auto;
      transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
        scale(0.8);
    }
  }
  #center {
    display: flex;
    flex-direction: column;
    gap: 25px;
    place-content: center;
    place-items: center;
    flex-grow: 1;
    @media (max-width: 1024px) {
      padding: 32px 20px 24px;
      gap: 18px;
    }
  }
  #next-steps {
    display: flex;
    border-top: 1px solid var(--border);
    text-align: left;
    & > div {
```

// ==================== [第 31 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
"""
feature_extractor.py
提取多肽序列的理化特征 (Physicochemical Descriptors) 与 ESM-2 嵌入特征。
完全自包含，无冗余外部依赖，支持轻量高效运行。
"""
import math
from typing import Dict, List, Tuple
import numpy as np
# 20 种标准天然氨基酸列表
STANDARD_AMINO_ACIDS = [
    'A', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'K', 'L',
    'M', 'N', 'P', 'Q', 'R', 'S', 'T', 'V', 'W', 'Y'
]
# Kyte-Doolittle 疏水性标度 (Hydropathy scale)
KYTE_DOOLITTLE = {
    'A': 1.8, 'C': 2.5, 'D': -3.5, 'E': -3.5, 'F': 2.8,
    'G': -0.4, 'H': -3.2, 'I': 4.5, 'K': -3.9, 'L': 3.8,
    'M': 1.9, 'N': -3.5, 'P': -1.6, 'Q': -3.5, 'R': -4.5,
    'S': -0.8, 'T': -0.7, 'V': 4.2, 'W': -0.9, 'Y': -1.3
}
# 氨基酸分子量 (Da)
MOLECULAR_WEIGHTS = {
    'A': 89.09, 'C': 121.16, 'D': 133.10, 'E': 147.13, 'F': 165.19,
    'G': 75.07, 'H': 155.16, 'I': 131.18, 'K': 146.19, 'L': 131.18,
    'M': 149.21, 'N': 132.12, 'P': 115.13, 'Q': 146.15, 'R': 174.20,
    'S': 105.09, 'T': 119.12, 'V': 117.15, 'W': 204.23, 'Y': 181.19
}
# 氨基酸 pKa 值与荷电倾向 (用于估计 pH 7.4 净电荷)
# 正电荷: K, R, H (部分); 负电荷: D, E
CHARGE_ESTIMATE = {
    'K': 1.0, 'R': 1.0, 'H': 0.1,
    'D': -1.0, 'E': -1.0
}
# Boman 指数 (蛋白质结合潜力指标, kcal/mol)
BOMAN_INDEX_MAP = {
    'L': -4.92, 'I': -4.92, 'V': -4.04, 'F': -2.98, 'W': -2.33,
    'A': -1.81, 'M': -1.45, 'C': -1.28, 'G': -0.94, 'Y': -0.14,
    'T': 2.57, 'S': 3.40, 'H': 4.66, 'Q': 5.54, 'K': 5.55,
    'N': 6.64, 'E': 6.81, 'D': 8.72, 'R': 9.38, 'P': 0.0
}
def clean_sequence(seq: str) -> str:
    """Normalize whitespace/case only; never silently edit molecular identity."""
    if not isinstance(seq, str):
        raise ValueError('Sequence must be a string.')
    cleaned = ''.join(seq.split()).upper()
    if not 3 <= len(cleaned) <= 1022:
        raise ValueError('Sequence length must be between 3 and 1022 residues.')
    invalid = set(cleaned) - set(STANDARD_AMINO_ACIDS)
    if invalid:
        raise ValueError('Noncanonical residues or non-sequence characters are not supported.')
    return cleaned
def compute_amino_acid_composition(seq: str) -> np.ndarray:
```

```
// ==================== [第 32 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
    """计算 20 维氨基酸组成频率 (Amino Acid Composition, AAC)"""
    length = len(seq)
    counts = {aa: 0 for aa in STANDARD_AMINO_ACIDS}
    for aa in seq:
        if aa in counts:
            counts[aa] += 1
    return np.array([counts[aa] / length for aa in STANDARD_AMINO_ACIDS], dtype=np.float32)
def compute_physicochemical_properties(seq: str) -> np.ndarray:
    """
    计算多肽核心物理化学特征 (8 维关键特征):
    1. 序列长度 (Length)
    2. 平均疏水性 (Mean Hydrophobicity)
    3. 净电荷 (Net Charge at pH 7.4)
    4. 电荷密度 (Charge Density)
    5. 总分子量 (Molecular Weight / 1000)
    6. 疏水残基比率 (Hydrophobic Ratio)
    7. 阳离子残基比率 (Cationic Ratio: K, R)
    8. Boman 指数 (结合潜力)
    """
    length = len(seq)
    # 疏水性统计
    hydro_values = [KYTE_DOOLITTLE.get(aa, 0.0) for aa in seq]
    mean_hydro = float(np.mean(hydro_values))
    hydrophobic_count = sum(1 for aa in seq if KYTE_DOOLITTLE.get(aa, 0.0) > 0.0)
    hydro_ratio = hydrophobic_count / length
    # 净电荷与阳离子比率
    net_charge = sum(CHARGE_ESTIMATE.get(aa, 0.0) for aa in seq)
    charge_density = net_charge / length
    cationic_count = sum(1 for aa in seq if aa in ['K', 'R'])
    cationic_ratio = cationic_count / length
    # 分子量 (减去脱水缩合产生的 (n-1)*18.0152 Da)
    mw = sum(MOLECULAR_WEIGHTS.get(aa, 110.0) for aa in seq) - (length - 1) * 18.0152
    mw_scaled = mw / 1000.0
    # Boman Index
    boman_idx = sum(BOMAN_INDEX_MAP.get(aa, 0.0) for aa in seq) / length
    features = [
        float(length) / 50.0,    # 归一化长度 (常见短肽 10~50)
        mean_hydro / 4.5,        # 归一化疏水性
        net_charge / 10.0,       # 归一化电荷
        charge_density,          # 电荷密度
        mw_scaled / 5.0,         # 归一化分子量
        hydro_ratio,             # 疏水残基比例
        cationic_ratio,          # 阳离子残基比例
        boman_idx / 5.0          # Boman 结合指数
    ]
    return np.array(features, dtype=np.float32)
def compute_dipeptide_top_features(seq: str, top_k: int = 16) -> np.ndarray:
    """
    提取高频关键二肽组成 (Dipeptide Composition, DPC)
    在抗菌肽中，带正电氨基酸与疏水氨基酸的交替出现 (如 KL, LK, RR, FL, LA) 是重要特征。
    """
    key_pairs = [
```

```
// ==================== [第 33 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
        'KL', 'LK', 'KK', 'LL', 'FL', 'LF', 'FA', 'AF',
        'KR', 'RK', 'IL', 'LI', 'AL', 'LA', 'GV', 'VG'
    ]
    if len(seq) < 2:
        return np.zeros(len(key_pairs), dtype=np.float32)
    total_pairs = len(seq) - 1
    pair_counts = {p: 0 for p in key_pairs}
    for i in range(total_pairs):
        pair = seq[i:i+2]
        if pair in pair_counts:
            pair_counts[pair] += 1
    return np.array([pair_counts[p] / total_pairs for p in key_pairs], dtype=np.float32)
def extract_handcrafted_features(seq: str) -> np.ndarray:
    """
    提取全部手工先验特征:
    AAC (20维) + 理化属性 (8维) + 关键二肽 (16维) = 共 44 维生物先验特征
    """
    cleaned = clean_sequence(seq)
    aac = compute_amino_acid_composition(cleaned)
    phys = compute_physicochemical_properties(cleaned)
    dpc = compute_dipeptide_top_features(cleaned)
    return np.concatenate([aac, phys, dpc])
class ESMFeatureExtractor:
    """Official fair-esm encoder. Synthetic fallback is explicit and test-only."""
    def __init__(self, model_name: str = 'esm2_t6_8M_UR50D', embedding_dim: int = 320, allow_fallback: bool =
False):
        if model_name != 'esm2_t6_8M_UR50D' or embedding_dim != 320:
            raise ValueError('Only the declared ESM-2 8M / 320-dimensional encoder is supported.')
        self.allow_fallback = allow_fallback
        self.feature_version = 'esm2-t6-8M-phys44-v2'
        self.model_name, self.embedding_dim = model_name, embedding_dim
        self.real_esm_available, self.backend = False, 'synthetic-test-only'
        self.model = self.tokenizer = self.batch_converter = None
        self.load_errors = []
        try:
            import esm
            self.model, self.alphabet = esm.pretrained.esm2_t6_8M_UR50D()
            self.model.eval()
            self.batch_converter = self.alphabet.get_batch_converter()
            self.real_esm_available, self.backend = True, 'fair-esm'
        except Exception as exc:
            self.load_errors.append(type(exc).__name__)
            if not allow_fallback:
                raise RuntimeError('Real ESM-2 encoder unavailable; formal inference is disabled.') from exc
    def extract_sequence_embedding(self, seq: str) -> Tuple[np.ndarray, np.ndarray]:
        """
        返回:
        - pooled_embedding: 整体序列的池化特征向量 (embedding_dim 维)
        - residue_embeddings: 每个残基的特征矩阵 (L x embedding_dim)
        """
        cleaned = clean_sequence(seq)
        L = len(cleaned)
        if self.real_esm_available and self.backend == "transformers":
```

// ==================== [第 34 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
            import torch
            inputs = self.tokenizer(cleaned, return_tensors="pt")
            with torch.no_grad():
                outputs = self.model(**inputs)
                # outputs.last_hidden_state: (1, L + 2, 320)
                # 去除 <cls> (index 0) 和 <eos> (index L+1)
                token_representations = outputs.last_hidden_state[0, 1:L+1, :].cpu().numpy()
                pooled = token_representations.mean(axis=0)
                return pooled.astype(np.float32), token_representations.astype(np.float32)
        elif self.real_esm_available and self.backend == "fair-esm":
            import torch
            data = [("protein", cleaned)]
            batch_labels, batch_strs, batch_tokens = self.batch_converter(data)
            with torch.no_grad():
                results = self.model(batch_tokens, repr_layers=[6], return_contacts=False)
                token_representations = results["representations"][6][0, 1:L+1].detach().cpu().numpy()
                pooled = token_representations.mean(axis=0)
                return pooled.astype(np.float32), token_representations.astype(np.float32)
        else:
            if not self.allow_fallback:
                raise RuntimeError('Synthetic embeddings are forbidden in research mode.')
            # 确定性仿生伪嵌入（基于氨基酸位置演化与理化先验矩阵生成 320 维特征）
            # 保证相同序列输出严格一致，同时保持语义距离与理化相关性
            rng = np.random.RandomState(seed=sum(ord(c) * (i + 1) for i, c in enumerate(cleaned)) % 1000000007)
            residue_embeddings = []
            handcrafted = extract_handcrafted_features(cleaned)
            # 使用氨基酸理化和 AAC 投射到 320 维
            proj_matrix = rng.randn(len(handcrafted), self.embedding_dim) * 0.1
            base_repr = np.dot(handcrafted, proj_matrix)
            for i, aa in enumerate(cleaned):
                pos_weight = math.sin((i + 1) / (L + 1) * math.pi)
                hydro = KYTE_DOOLITTLE.get(aa, 0.0)
                charge = CHARGE_ESTIMATE.get(aa, 0.0)
                res_vec = base_repr + pos_weight * 0.2 + hydro * 0.15 + charge * 0.25
                residue_embeddings.append(res_vec)
            res_mat = np.array(residue_embeddings, dtype=np.float32)
            pooled = res_mat.mean(axis=0)
            return pooled, res_mat
if __name__ == "__main__":
    test_seq = "GIGAVLKVLTTGLPALISWIKRKRQQ"  # 蜂毒素 Melittin
    print(f"测试序列: {test_seq}")
    phys_feat = extract_handcrafted_features(test_seq)
    print(f"手工理化特征维度: {phys_feat.shape} (预期 44 维)")
    extractor = ESMFeatureExtractor()
    pooled_emb, res_emb = extractor.extract_sequence_embedding(test_seq)
    print(f"ESM 嵌入特征维度: {pooled_emb.shape} (预期 320 维)")
    print(f"残基级嵌入矩阵维度: {res_emb.shape} (预期 {len(test_seq)} x 320)")
    print("特征提取器功能验证成功！")
"""
helical_wheel.py
计算基于 Schiffer-Edmundson 100° 投影模型的 α-螺旋轮构象数据与 Eisenberg 疏水力矩。
为 SafeAMP-Dual 3.0 工作站提供高精度物理化学与几何投影支持。
```

// ==================== [第 35 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
"""
import os
import sys
import math
from typing import Dict, List, Tuple, Any
import numpy as np
# 确保能正确引用当前目录下的 feature_extractor 模块
_CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
if _CURRENT_DIR not in sys.path:
    sys.path.insert(0, _CURRENT_DIR)
from feature_extractor import clean_sequence, KYTE_DOOLITTLE, CHARGE_ESTIMATE
def get_residue_category(aa: str) -> Tuple[str, str]:
    """
    根据 Kyte-Doolittle 疏水性与电荷倾向返回氨基酸生信色彩分类：
    - basic: 碱性阳离子 (天蓝 #38bdf8)
    - acidic: 酸性阴离子 (玫红 #f43f5e)
    - hydrophobic: 强疏水 (橙黄 #f59e0b)
    - moderate: 中度疏水 (淡黄 #fbbf24)
    - polar: 极性中性 (草绿 #10b981)
    Returns:
        (category_name, hex_color)
    """
    charge = CHARGE_ESTIMATE.get(aa, 0.0)
    hydro = KYTE_DOOLITTLE.get(aa, 0.0)
    if charge > 0:
        return "basic", "#38bdf8"
    elif charge < 0:
        return "acidic", "#f43f5e"
    elif hydro >= 1.8:
        return "hydrophobic", "#f59e0b"
    elif hydro > 0:
        return "moderate", "#fbbf24"
    else:
        return "polar", "#10b981"
def compute_helical_wheel_data(seq: str, radius: float = 120.0) -> Dict[str, Any]:
    """
    计算基于 Schiffer-Edmundson 100 度投影模型的螺旋轮构象数据及 Eisenberg 两亲疏水力矩。
    Args:
        seq: 多肽单字母序列
        radius: 螺旋轮投影半径 (默认 120.0，中心坐标为 (160.0, 160.0))
    Returns:
        Dict 包含:
            - sequence: 清理后的大写序列
            - length: 序列长度
            - residues: 残基投影列表 (pos, aa, angle_deg, x, y, category, color, hydrophobicity, charge)
            - hydrophobic_moment: Eisenberg 疏水力矩模长 (μH)
            - moment_angle_deg: 疏水力矩指向角度 (0~360°, 12 点钟为 0°, 顺时针)
            - hydrophobic_angle_span: 疏水面跨度角度
            - net_charge: pH 7.4 净电荷估计
            - mean_hydrophobicity: 平均 Kyte-Doolittle 疏水性
    """
    cleaned = clean_sequence(seq)
```

// ==================== [第 36 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
    L = len(cleaned)
    residues: List[Dict[str, Any]] = []
    sum_x = 0.0
    sum_y = 0.0
    hydro_angles = []
    for idx, aa in enumerate(cleaned):
        # Schiffer-Edmundson 投影：每个残基在圆周上顺时针步进 100°
        angle_deg = (idx * 100.0) % 360.0
        angle_rad = math.radians(angle_deg)
        # 极坐标转换为笛卡尔坐标（以 (160.0, 160.0) 为圆心，12 点钟方向为 0°）
        cx = 160.0 + radius * math.sin(angle_rad)
        cy = 160.0 - radius * math.cos(angle_rad)
        category, color = get_residue_category(aa)
        hydro = KYTE_DOOLITTLE.get(aa, 0.0)
        charge = CHARGE_ESTIMATE.get(aa, 0.0)
        # 归一化疏水性贡献向量（KD 标度从 -4.5 到 4.5 归一化至 0.0 ~ 1.0）
        h_norm = (hydro + 4.5) / 9.0
        sum_x += h_norm * math.sin(angle_rad)
        sum_y += -h_norm * math.cos(angle_rad)
        if hydro > 0:
            hydro_angles.append(angle_deg)
        residues.append({
            "pos": idx + 1,
            "aa": aa,
            "angle_deg": round(angle_deg, 1),
            "x": round(cx, 1),
            "y": round(cy, 1),
            "category": category,
            "color_category": category,
            "color": color,
            "hydrophobicity": float(hydro),
            "charge": float(charge)
        })
    # Eisenberg 疏水力矩模长
    moment_mag = math.sqrt(sum_x ** 2 + sum_y ** 2) / (L + 1e-6)
    # 疏水力矩方向（以 12 点钟为 0°，顺时针 0~360°）
    moment_angle_deg = (math.degrees(math.atan2(sum_x, -sum_y)) + 360.0) % 360.0
    # 计算疏水面角度跨度（Hydrophobic arc span）
    if len(hydro_angles) >= 2:
        # 计算所有疏水残基相对于力矩矢量的角偏离
        diffs = [abs((a - moment_angle_deg + 180.0) % 360.0 - 180.0) for a in hydro_angles]
        hydro_span = min(360.0, 2.0 * max(diffs))
    elif len(hydro_angles) == 1:
        hydro_span = 20.0
    else:
        hydro_span = 0.0
    net_charge = sum(CHARGE_ESTIMATE.get(aa, 0.0) for aa in cleaned)
    mean_hydro = float(np.mean([KYTE_DOOLITTLE.get(aa, 0.0) for aa in cleaned]))
    return {
        "sequence": cleaned,
        "length": L,
        "residues": residues,
```

// ==================== [第 37 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
            "hydrophobic_moment": round(moment_mag, 4),
            "moment_angle_deg": round(moment_angle_deg, 1),
            "hydrophobic_angle_span": round(hydro_span, 1),
            "net_charge": round(net_charge, 2),
            "mean_hydrophobicity": round(mean_hydro, 2)
        }
    """Exploratory sequence ranking and fixed physicochemical property displays.
    No function in this module establishes clinical safety or experimental efficacy.
    """
    import sys
    import copy
    from typing import List, Dict, Tuple, Optional
    import numpy as np
    import torch
    from feature_extractor import (
        clean_sequence, extract_handcrafted_features, ESMFeatureExtractor,
        STANDARD_AMINO_ACIDS, KYTE_DOOLITTLE, CHARGE_ESTIMATE, BOMAN_INDEX_MAP
    )
    from model_dual_task import SafeAMPDualNet
    class AIPeptideOptimizer:
        def __init__(self, model: SafeAMPDualNet, extractor: ESMFeatureExtractor, device: str = "cpu"):
            self.model = model
            self.extractor = extractor
            self.device = torch.device(device)
            self.model.to(self.device)
            self.model.eval()
        def evaluate_sequence(self, seq: str) -> Tuple[float, float, float]:
            """评估单条序列的 P_AMP, P_Hemo, TSI"""
            cleaned = clean_sequence(seq)
            p_emb, _ = self.extractor.extract_sequence_embedding(cleaned)
            p_phys = extract_handcrafted_features(cleaned)
            t_esm = torch.tensor(p_emb, device=self.device).unsqueeze(0)
            t_phys = torch.tensor(p_phys, device=self.device).unsqueeze(0)
            with torch.no_grad():
                p_amp, p_hemo, tsi = self.model.predict_probabilities(t_esm, t_phys)
            return float(p_amp[0]), float(p_hemo[0]), float(tsi[0])
        def compute_residue_attributions(self, seq: str) -> List[Dict]:
            """Compatibility method name: fixed properties, never model attribution."""
            return [{'pos': i + 1, 'aa': aa, 'hydrophobicity': KYTE_DOOLITTLE[aa],
                     'charge': CHARGE_ESTIMATE.get(aa, 0.),
                     'hydrophobicity_band': 'high' if KYTE_DOOLITTLE[aa] > 2.5 else 'positive' if KYTE_DOOLITTLE[aa]
    > 0 els
                    for i, aa in enumerate(clean_sequence(seq))]
        def optimize_mutations(self, seq: str, top_k: int = 3) -> List[Dict]:
            """
            单点候选搜索；固定概率筛选条件仅用于探索性计算排序。
            """
            cleaned = clean_sequence(seq)
            orig_amp, orig_hemo, orig_tsi = self.evaluate_sequence(cleaned)
            # Prespecified substitutions at hydrophobic sites; no mechanistic inference.
            substitute_candidates = ['K', 'R', 'A', 'S', 'G']
            tested_variants = []
            # 1. 单点突变扫描 (Single Point Mutation)
```

```
// ==================== [第 38 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
        for i, original_aa in enumerate(cleaned):
            # 优先扫描中高疏水性的位点
            if KYTE_DOOLITTLE.get(original_aa, 0.0) <= 0.5 and original_aa not in ['L', 'F', 'I', 'V', 'W']:
                continue
            for sub_aa in substitute_candidates:
                if sub_aa == original_aa:
                    continue
                mut_seq = cleaned[:i] + sub_aa + cleaned[i+1:]
                m_amp, m_hemo, m_tsi = self.evaluate_sequence(mut_seq)
                # Score-only constraints; 0.85 does not mean 85% measured killing.
                if m_amp >= 0.85 and m_hemo < orig_hemo and m_tsi > orig_tsi:
                    gain = m_tsi - orig_tsi
                    tested_variants.append({
                        "mutation": f"{original_aa}{i+1}{sub_aa}",
                        "pos": i + 1,
                        "original_aa": original_aa,
                        "mutated_aa": sub_aa,
                        "mutated_seq": mut_seq,
                        "amp_prob": round(m_amp, 4),
                        "hemo_prob": round(m_hemo, 4),
                        "tsi": round(m_tsi, 4),
                        "tsi_gain": round(gain, 4),
                        "hemo_probability_change": round(m_hemo - orig_hemo, 4),
                        "rationale": "该候选仅改善当前模型的计算排序分数；不代表实测降毒、活性保留或膜机制已得到证实。",
                        "validation_status": "exploratory_same_model_scoring"
                    })
        # 按 TSI 收益降序排列
        tested_variants.sort(key=lambda x: x["tsi"], reverse=True)
        return tested_variants[:top_k]
    def generate_scientific_report(self, seq: str) -> Dict:
        """Return measured-source model scores without clinical or mechanistic inference."""
        import hashlib
        cleaned = clean_sequence(seq)
        a, h, score = self.evaluate_sequence(cleaned)
        attrs = self.compute_residue_attributions(cleaned)
        report_id = 'RESEARCH-' + hashlib.sha256(cleaned.encode()).hexdigest()[:12]
        metrics = {'amp_probability': a, 'hemo_probability': h,
                   'ranking_score': score, 'therapeutic_safety_index': score,
                   'net_charge': float(sum(CHARGE_ESTIMATE.get(x, 0.) for x in cleaned)),
                   'mean_hydrophobicity': float(np.mean([KYTE_DOOLITTLE[x] for x in cleaned])),
                   'high_hydrophobicity_residue_count': sum(x['hydrophobicity_band']=='high' for x in attrs),
                   'positive_hydrophobicity_residue_count': sum(x['hydrophobicity_band']=='positive' for x in
attrs),
                   'nonpositive_hydrophobicity_residue_count': sum(x['hydrophobicity_band']=='nonpositive' for x
in attr
        text = (f'# Computational research report {report_id}\n\n'
                f'AMP endpoint score: {a:.6f}\nHemolysis endpoint score: {h:.6f}\n'
                f'Exploratory ranking score: {score:.6f}\n\n'
                'Scores refer to the declared model endpoints, not measured killing rates, '
                'clinical safety, therapeutic windows, or mechanisms. Residue coloring is '
                'a fixed physicochemical heuristic, not model gradient attribution. '
                'No laboratory experiment was performed by this service. '
                'Same-model candidate scoring is not independent validation.')
        return {'report_id': report_id, 'protocol_version': 'research-v1',
```

```
// ==================== [第 39 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
                'sequence': cleaned, 'length': len(cleaned), 'metrics': metrics,
                'clinical_tier': 'Not clinically validated', 'tier_code': 'Research-only',
                'disposition': '计算结果待独立验证', 'disposition_badge': 'warning',
                'mechanism_model': 'Not established',
                'mechanism_deduction': '理化提示不能证明膜作用机制或临床安全性。',
                'residue_attributions': attrs, 'attribution_type': 'physicochemical_heuristic',
                'ai_mutation_solutions': [], 'standard_markdown_report': text}
if __name__ == '__main__':
    import os
    from inference_runtime import ResearchRuntime
    runtime = ResearchRuntime(os.environ.get('SAFEAMP_RUN_DIR')).load()
    optimizer = AIPeptideOptimizer(runtime, runtime.extractor)
    print(optimizer.generate_scientific_report('GIGAVLKVLTTGLPALISWIKRKRQQ')['standard_markdown_report'])
"""Asynchronous batch task manager for high-throughput prediction jobs."""
import threading
import uuid
from concurrent.futures import ThreadPoolExecutor
from datetime import datetime, timezone
from enum import Enum
from typing import Any, Callable, Dict, List, Optional
class BatchJobStatus(str, Enum):
    QUEUED = "queued"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    CANCELLED = "cancelled"
class BatchTaskManager:
    """Manages thread-safe asynchronous execution of batch prediction jobs."""
    def __init__(self, max_workers: int = 4):
        self.max_workers = max_workers
        self._executor = ThreadPoolExecutor(max_workers=max_workers)
        self._lock = threading.Lock()
        self._jobs: Dict[str, Dict[str, Any]] = {}
    @staticmethod
    def _current_timestamp() -> str:
        return datetime.now(timezone.utc).isoformat()
    def submit_job(self, sequences: List[str], worker_fn: Callable[[str], Any]) -> str:
        job_id = f"job-{uuid.uuid4().hex[:12]}"
        now = self._current_timestamp()
        with self._lock:
            self._jobs[job_id] = {
                "job_id": job_id,
                "status": BatchJobStatus.QUEUED.value,
                "total": len(sequences),
                "completed": 0,
                "failed_count": 0,
                "results": [],
                "errors": [],
                "created_at": now,
                "updated_at": now,
                "_cancel_flag": False,
                "_sequences": list(sequences),
```

```
// ==================== [第 40 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
                "_worker_fn": worker_fn,
            }
        self._executor.submit(self._execute_job, job_id)
        return job_id
    def _execute_job(self, job_id: str) -> None:
        with self._lock:
            job = self._jobs.get(job_id)
            if job is None:
                return
            if job["_cancel_flag"]:
                job["status"] = BatchJobStatus.CANCELLED.value
                job["updated_at"] = self._current_timestamp()
                return
            job["status"] = BatchJobStatus.RUNNING.value
            job["updated_at"] = self._current_timestamp()
            sequences = list(job["_sequences"])
            worker_fn = job["_worker_fn"]
        try:
            for idx, seq in enumerate(sequences):
                with self._lock:
                    if job["_cancel_flag"]:
                        job["status"] = BatchJobStatus.CANCELLED.value
                        job["updated_at"] = self._current_timestamp()
                        return
                try:
                    res = worker_fn(seq)
                    with self._lock:
                        if job["_cancel_flag"]:
                            job["status"] = BatchJobStatus.CANCELLED.value
                            job["updated_at"] = self._current_timestamp()
                            return
                        job["results"].append(res)
                        job["completed"] += 1
                        job["updated_at"] = self._current_timestamp()
                except Exception as exc:
                    with self._lock:
                        if job["_cancel_flag"]:
                            job["status"] = BatchJobStatus.CANCELLED.value
                            job["updated_at"] = self._current_timestamp()
                            return
                        job["errors"].append({
                            "index": idx,
                            "sequence": seq[:15],
                            "error": str(exc),
                        })
                        job["failed_count"] += 1
                        job["completed"] += 1
                        job["updated_at"] = self._current_timestamp()
            with self._lock:
                if job["_cancel_flag"]:
                    job["status"] = BatchJobStatus.CANCELLED.value
                else:
```

```
// ==================== [第 41 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
                    job["status"] = BatchJobStatus.COMPLETED.value
                job["updated_at"] = self._current_timestamp()
        except Exception as fatal_exc:
            with self._lock:
                job["status"] = BatchJobStatus.FAILED.value
                job["errors"].append({
                    "index": -1,
                    "sequence": "",
                    "error": f"Fatal execution error: {str(fatal_exc)}",
                })
                job["updated_at"] = self._current_timestamp()
    def get_status(self, job_id: str) -> Optional[Dict[str, Any]]:
        with self._lock:
            job = self._jobs.get(job_id)
            if job is None:
                return None
            return {
                k: (list(v) if isinstance(v, list) else v)
                for k, v in job.items()
                if not k.startswith("_")
            }
    def cancel_job(self, job_id: str) -> bool:
        with self._lock:
            job = self._jobs.get(job_id)
            if job is None:
                return False
            job["_cancel_flag"] = True
            job["status"] = BatchJobStatus.CANCELLED.value
            job["updated_at"] = self._current_timestamp()
            return True
    def shutdown(self, wait: bool = True) -> None:
        self._executor.shutdown(wait=wait)
"""Lazy, fail-closed loading of a completed, provenance-validated ensemble."""
from pathlib import Path
import numpy as np
import torch
from scipy.special import expit, logit
from feature_extractor import ESMFeatureExtractor
from model_dual_task import SafeAMPDualNet
from research_utils import read_json, sha256
class ModelUnavailable(RuntimeError):
    pass
class ResearchRuntime:
    def __init__(self,run_dir=None):
        self.run_dir=Path(run_dir) if run_dir else None
        self.extractor=None;self.members=[];self.metadata={};self.error=None
    def load(self):
        if self.members:return self
        try:
            if self.run_dir is None:raise ModelUnavailable('No validated research run configured
(SAFEAMP_RUN_DIR).')
            manifest=read_json(self.run_dir/'manifest.json')
            if manifest.get('status')!='completed' or manifest.get('dataset_manifest',{}).get('status')!
='validated':
```

```
// ==================== [第 42 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
                    raise ModelUnavailable('Research run or dataset did not pass validation.')
                if sha256(self.run_dir/'predictions.csv')!=manifest.get('predictions_sha256'):
                    raise ModelUnavailable('Prediction artifact checksum mismatch.')
                checkpoints=manifest.get('checkpoints',[])
                if len(checkpoints)!=3 or {c['seed'] for c in checkpoints}!={11,23,42}:
                    raise ModelUnavailable('Complete three-seed ensemble required.')
                for c in checkpoints:
                    path=(self.run_dir/c['path']).resolve()
                    if not path.is_relative_to(self.run_dir.resolve()) or sha256(path)!=c['sha256']:
                        raise ModelUnavailable('Model checkpoint checksum mismatch.')
                for module in ['feature_extractor.py','model_dual_task.py']:
                    if manifest.get('code_hashes',{}).get(module)!=sha256(Path(__file__).parent/module):
                        raise ModelUnavailable('Feature or architecture implementation changed since training.')
                extractor=ESMFeatureExtractor()
                feature=manifest['features']
                esm_file=Path(torch.hub.get_dir())/'checkpoints/esm2_t6_8M_UR50D.pt'
                if (not extractor.real_esm_available or extractor.backend!=feature['backend']
                    or extractor.feature_version!=feature['version'] or sha256(esm_file)!=feature['esm_sha256']):
                    raise ModelUnavailable('Feature encoder provenance mismatch.')
                members=[]
                for c in checkpoints:
                    ck=torch.load(self.run_dir/c['path'],map_location='cpu',weights_only=True)
                    if ck['features']!=feature or ck['seed']!=c['seed']:
                        raise ModelUnavailable('Checkpoint metadata mismatch.')
                    config=ck['config']
                    if config!=manifest['config']:
                        raise ModelUnavailable('Checkpoint configuration differs from run manifest.')
                    for prefix,size in [('esm',320),('phys',44)]:
                        mean,scale=np.asarray(ck[prefix+'_mean']),np.asarray(ck[prefix+'_scale'])
                        if mean.shape!=(size,) or scale.shape!=(size,) or not np.isfinite(mean).all() or not
np.isfinite(sca
                            raise ModelUnavailable('Invalid saved preprocessing parameters.')
                    if len(ck['calibration'])!=2 or not all(np.isfinite([v['coefficient'],v['intercept']]).all() for
v in ck
                        raise ModelUnavailable('Invalid saved calibration parameters.')
                    model=SafeAMPDualNet(hidden_dim=config['hidden_dim'],dropout_rate=config['dropout'])
                    model.load_state_dict(ck['state_dict'],strict=True);model.eval()
                    if any(not torch.isfinite(p).all() for p in model.parameters()):
                        raise ModelUnavailable('Non-finite model parameters.')
                    members.append((model,ck))
                self.members=members;self.extractor=extractor

self.metadata={'run_id':self.run_dir.name,'backend':feature['backend'],'feature_version':feature['version'],
                               'seeds':
[11,23,42],'score_type':'computational_ranking_score','clinical_validation':False,
                               'model_kind':'mean_of_three_calibrated_seed_predictions',
                               'amp_endpoint':'E. coli MIC <=32 vs >=128 ug/ml','hemo_endpoint':'Human erythrocyte
HC50 <=10
                self.error=None
            except Exception as exc:
                self.error=str(exc);self.members=[];self.extractor=None
                raise ModelUnavailable(str(exc)) from exc
            return self
        def status(self):
            try:self.load()
            except ModelUnavailable:pass
            ready=bool(self.members)
```

```
// ==================== [第 43 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
        return {'status':'healthy' if ready else 'unavailable','reason':self.error,
                'esm_model':{'name':'ESM-2
8M','backend':self.metadata.get('backend','unavailable'),'status':'Loaded' if
                'dual_task_model':{'name':'SafeAMP-Dual research
ensemble','weights_loaded':ready,'status':'Validated' i
                'mutation_engine':{'name':'Exploratory scoring','status':'Ready' if ready else
'Disabled'},'provenance':
    def to(self,device):return self
    def eval(self):return self
    @torch.no_grad()
    def predict_probabilities(self,x_esm,x_phys):
        self.load()
        e=x_esm.detach().cpu().numpy();p=x_phys.detach().cpu().numpy();values=[]
        for model,ck in self.members:
            es=(e-np.asarray(ck['esm_mean']))/np.asarray(ck['esm_scale'])
            ph=(p-np.asarray(ck['phys_mean']))/np.asarray(ck['phys_scale'])
            a,h,_=model(torch.tensor(es,dtype=torch.float32),torch.tensor(ph,dtype=torch.float32))
            raw=np.column_stack([torch.sigmoid(a).numpy(),torch.sigmoid(h).numpy()])
            for t,c in enumerate(ck['calibration']):
                raw[:,t]=expit(c['coefficient']*logit(np.clip(raw[:,t],1e-6,1-1e-6))+c['intercept'])
            values.append(raw)
        mean=np.mean(values,axis=0)
        return mean[:,0],mean[:,1],mean[:,0]*(1-mean[:,1])
"""Unified biological assay measurement representation and label derivation.
Separates raw measurements from discrete labels:
- measurement_present: Whether an assay record exists.
- endpoint_eligible: Whether target species/cell, metric, and units match criteria.
- label_observed: Whether a definitive binary label is derived.
Unknown reasons:
- not_measured
- incompatible_target
- incompatible_endpoint
- unsupported_unit
- ambiguous_censoring
- intermediate_range
- conflicting_measurements
- unsupported_modification
- unresolved_evidence
"""
from dataclasses import dataclass, field, asdict
import json
import math
from pathlib import Path
import re
from typing import Optional, Any
from feature_extractor import MOLECULAR_WEIGHTS, STANDARD_AMINO_ACIDS, clean_sequence
from research_utils import sha256
POSITIVE_CUTOFF_AMP = 32.0   # E. coli MIC <= 32 ug/ml
NEGATIVE_CUTOFF_AMP = 128.0  # E. coli MIC >= 128 ug/ml
POSITIVE_CUTOFF_HEMO = 100.0  # Human erythrocytes HC50 <= 100 ug/ml
NEGATIVE_CUTOFF_HEMO = 250.0  # Human erythrocytes HC50 >= 250 ug/ml
def parse_measurement_bounds(value_str: Any) -> Optional[tuple[float, float, str]]:
    """Parse numeric concentration bounds and operator.
    Returns (low, high, operator_type) or None if unparseable.
    operator_type in: 'exact', 'less_equal', 'greater_equal', 'interval'
```

// ==================== [第 44 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
    """
    if value_str is None:
        return None
    text = str(value_str).strip().replace('≤', '<=').replace('≥', '>=')
    number = r'(?:\d+(?:\.\d*)?|\.\d+)(?:[eE][+-]?\d+)?'
    # Check operator + number: <= X, < X, >= X, > X, = X, X
    m = re.fullmatch(r'([<>]=?|=)?\s*(' + number + r')', text)
    if m:
        op = m.group(1) or '='
        try:
            x = float(m.group(2))
        except ValueError:
            return None
        if x <= 0 or math.isnan(x) or math.isinf(x):
            return None
        if op in ('<=', '<'):
            return 0.0, x, 'less_equal'
        if op in ('>=', '>'):
            return x, math.inf, 'greater_equal'
        return x, x, 'exact'
    # Check interval: X - Y or X – Y
    m_int = re.fullmatch(r'(' + number + r')\s*[--]\s*(' + number + r')', text)
    if m_int:
        try:
            low, high = float(m_int.group(1)), float(m_int.group(2))
        except ValueError:
            return None
        if 0 < low <= high and not (math.isnan(low) or math.isnan(high) or math.isinf(low) or math.isinf(high)):
            return low, high, 'interval'
    return None
def calculate_mw(sequence: str) -> float:
    """Calculate monoisotopic molecular weight of canonical linear peptide."""
    return sum(MOLECULAR_WEIGHTS[aa] for aa in sequence) - 18.0152 * (len(sequence) - 1)
def normalize_concentration_to_ug_ml(low: float, high: float, unit_str: str, sequence: str) ->
Optional[tuple[float, flo
    """Convert concentration to ug/mL based on sequence molecular weight."""
    u = unit_str.replace('μ', 'u').replace('µ', 'u').lower().replace(' ', '')
    if u in ('ug/ml', 'mg/l', 'mcg/ml'):
        factor = 1.0
    elif u in ('um', 'micromolar', 'umol/l'):
        mw = calculate_mw(sequence)
        factor = mw / 1000.0  # (umol / L) * (g / mol) = ug / mL
    else:
        return None
    high_conv = math.inf if math.isinf(high) else high * factor
    return low * factor, high_conv
@dataclass
class SingleAssayObservation:
    endpoint: str            # 'amp' or 'hemo'
    target_organism_or_cell: str
    metric_name: str         # 'MIC', 'HC50', etc.
    raw_value: str
    raw_unit: str
```

```
// ==================== [第 45 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
    conditions: str = ''
    reference: str = ''
    # Derived properties
    eligible: bool = False
    ineligibility_reason: str = ''
    normalized_low_ug_ml: Optional[float] = None
    normalized_high_ug_ml: Optional[float] = None
    derived_label: Optional[int] = None   # 1, 0, or None
@dataclass
class StandardizedMoleculeRecord:
    record_id: str
    sequence: str
    source_dataset: str
    source_accession: str
    source_url: str
    source_role: str = 'development'
    evidence_reference: str = ''
    modifications: str = 'none'
    evidence_type: str = 'experimental'
    is_synthetic: bool = False
    raw_path: str = ''
    raw_sha256: str = ''
    observations: list[SingleAssayObservation] = field(default_factory=list)
    # Aggregated binary labels
    amp_label: Optional[int] = None
    hemo_label: Optional[int] = None
    amp_unknown_reasons: list[str] = field(default_factory=list)
    hemo_unknown_reasons: list[str] = field(default_factory=list)
def evaluate_assay_for_amp(assay: SingleAssayObservation, sequence: str) -> None:
    """Evaluate observation against E. coli MIC criteria."""
    if not any(assay.target_organism_or_cell.lower().startswith(ec) for ec in ['escherichia coli', 'e. coli']):
        assay.eligible = False
        assay.ineligibility_reason = 'incompatible_target'
        return
    if assay.metric_name.upper() not in ('MIC', 'MINIMUM INHIBITORY CONCENTRATION'):
        assay.eligible = False
        assay.ineligibility_reason = 'incompatible_endpoint'
        return
    parsed = parse_measurement_bounds(assay.raw_value)
    if parsed is None:
        assay.eligible = False
        assay.ineligibility_reason = 'ambiguous_censoring'
        return
    low, high, op = parsed
    conv = normalize_concentration_to_ug_ml(low, high, assay.raw_unit, sequence)
    if conv is None:
        assay.eligible = False
        assay.ineligibility_reason = 'unsupported_unit'
        return
    norm_low, norm_high = conv
    assay.normalized_low_ug_ml = norm_low
    assay.normalized_high_ug_ml = norm_high
```

// ==================== [第 46 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
        assay.eligible = True
        # Cutoffs: <= 32 positive, >= 128 negative
        if norm_high <= POSITIVE_CUTOFF_AMP:
            assay.derived_label = 1
        elif norm_low >= NEGATIVE_CUTOFF_AMP:
            assay.derived_label = 0
        else:
            assay.derived_label = None
            assay.ineligibility_reason = 'intermediate_range'
    def evaluate_assay_for_hemo(assay: SingleAssayObservation, sequence: str) -> None:
        """Evaluate observation against Human Erythrocyte HC50 criteria."""
        target_clean = assay.target_organism_or_cell.lower()
        if 'human' not in target_clean or not any(rbc in target_clean for rbc in ['erythrocyte', 'red blood',
'rbc']):
            assay.eligible = False
            assay.ineligibility_reason = 'incompatible_target'
            return
        metric_clean = assay.metric_name.upper().replace(' ', '')
        if metric_clean not in ('HC50', '50%HEMOLYSIS', '50%HAEMOLYSIS'):
            assay.eligible = False
            assay.ineligibility_reason = 'incompatible_endpoint'
            return
        parsed = parse_measurement_bounds(assay.raw_value)
        if parsed is None:
            assay.eligible = False
            assay.ineligibility_reason = 'ambiguous_censoring'
            return
        low, high, op = parsed
        conv = normalize_concentration_to_ug_ml(low, high, assay.raw_unit, sequence)
        if conv is None:
            assay.eligible = False
            assay.ineligibility_reason = 'unsupported_unit'
            return
        norm_low, norm_high = conv
        assay.normalized_low_ug_ml = norm_low
        assay.normalized_high_ug_ml = norm_high
        assay.eligible = True
        # Cutoffs: <= 100 positive, >= 250 negative
        if norm_high <= POSITIVE_CUTOFF_HEMO:
            assay.derived_label = 1
        elif norm_low >= NEGATIVE_CUTOFF_HEMO:
            assay.derived_label = 0
        else:
            assay.derived_label = None
            assay.ineligibility_reason = 'intermediate_range'
    def process_molecule_record(record: StandardizedMoleculeRecord) -> tuple[Optional[dict], Optional[str]]:
        """Validate sequence, evaluate observations, aggregate labels, and check conflicts."""
        clean_seq = clean_sequence(record.sequence)
        if not (10 <= len(clean_seq) <= 50) or set(clean_seq) - set(STANDARD_AMINO_ACIDS):
            return None, 'noncanonical_or_length'
        record.sequence = clean_seq
        amp_labels = set()
        hemo_labels = set()
```

```
// ==================== [第 47 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
    amp_reasons = []
    hemo_reasons = []
    for obs in record.observations:
        if obs.endpoint == 'amp':
            evaluate_assay_for_amp(obs, clean_seq)
            if obs.derived_label is not None:
                amp_labels.add(obs.derived_label)
            elif obs.ineligibility_reason:
                amp_reasons.append(obs.ineligibility_reason)
        elif obs.endpoint == 'hemo':
            evaluate_assay_for_hemo(obs, clean_seq)
            if obs.derived_label is not None:
                hemo_labels.add(obs.derived_label)
            elif obs.ineligibility_reason:
                hemo_reasons.append(obs.ineligibility_reason)
    # Determine final labels
    if len(amp_labels) == 1:
        record.amp_label = next(iter(amp_labels))
    elif len(amp_labels) > 1:
        record.amp_label = None
        record.amp_unknown_reasons.append('conflicting_measurements')
    else:
        record.amp_label = None
        record.amp_unknown_reasons.extend(amp_reasons or ['not_measured'])
    if len(hemo_labels) == 1:
        record.hemo_label = next(iter(hemo_labels))
    elif len(hemo_labels) > 1:
        record.hemo_label = None
        record.hemo_unknown_reasons.append('conflicting_measurements')
    else:
        record.hemo_label = None
        record.hemo_unknown_reasons.extend(hemo_reasons or ['not_measured'])
    if record.amp_label is None and record.hemo_label is None:
        return None, 'missing_or_conflicting_endpoints'
    # Build row matching REQUIRED contract
    row = {
        'record_id': record.record_id,
        'seq': record.sequence,
        'amp': record.amp_label,
        'hemo': record.hemo_label,
        'source_dataset': record.source_dataset,
        'source_accession': record.source_accession,
        'source_url': record.source_url,
        'evidence_reference': record.evidence_reference,
        'label_basis': 'Ecoli_MIC_ugml_32_128;human_HC50_ugml_100_250;conflicts_missing',
        'assay_conditions': json.dumps([asdict(o) for o in record.observations if o.eligible],
ensure_ascii=False),
        'modifications': record.modifications,
        'evidence_type': record.evidence_type,
        'is_synthetic': record.is_synthetic,
        'source_role': record.source_role,
        'raw_path': record.raw_path,
        'raw_sha256': record.raw_sha256,
```

```
// ==================== [第 48 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
    }
    return row, None
"""Request tracking middleware and structured audit logging."""
import json
import threading
import time
import uuid
from pathlib import Path
from typing import Any, Dict, List, Optional
from starlette.middleware.base import BaseHTTPMiddleware, RequestResponseEndpoint
from starlette.requests import Request
from starlette.responses import Response
from research_utils import ROOT, utc_now
AUDIT_LOG_FILE = ROOT / "output" / "audit" / "api_audit.jsonl"
_file_lock = threading.Lock()
class AuditLogMiddleware(BaseHTTPMiddleware):
    """FastAPI/Starlette middleware for X-Request-ID propagation, latency timing,
    and structured JSONL audit logging.
    """
    def __init__(self, app, log_file: Optional[Path] = None):
        super().__init__(app)
        self._custom_log_file = Path(log_file) if log_file is not None else None
        self._lock = _file_lock
    @property
    def log_file(self) -> Path:
        if self._custom_log_file is not None:
            return self._custom_log_file
        # Allow dynamic monkeypatching of module-level AUDIT_LOG_FILE
        return AUDIT_LOG_FILE
    def _append_log(self, entry: Dict[str, Any]) -> None:
        target_file = self.log_file
        target_file.parent.mkdir(parents=True, exist_ok=True)
        line = json.dumps(entry, ensure_ascii=False) + "\n"
        with self._lock:
            with open(target_file, "a", encoding="utf-8") as f:
                f.write(line)
    async def dispatch(self, request: Request, call_next: RequestResponseEndpoint) -> Response:
        raw_id = request.headers.get("x-request-id")
        request_id = raw_id.strip() if raw_id and raw_id.strip() else f"req-{uuid.uuid4().hex[:12]}"
        start_time = time.perf_counter()
        client_ip = request.client.host if request.client else "unknown"
        try:
            response = await call_next(request)
            duration_ms = round((time.perf_counter() - start_time) * 1000, 2)
            response.headers["X-Request-ID"] = request_id
            log_entry = {
                "timestamp": utc_now(),
                "request_id": request_id,
                "method": request.method,
                "path": request.url.path,
                "status_code": response.status_code,
                "duration_ms": duration_ms,
```

```
// ==================== [第 49 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
                "client_ip": client_ip,
            }
            self._append_log(log_entry)
            return response
        except Exception as exc:
            duration_ms = round((time.perf_counter() - start_time) * 1000, 2)
            log_entry = {
                "timestamp": utc_now(),
                "request_id": request_id,
                "method": request.method,
                "path": request.url.path,
                "status_code": 500,
                "duration_ms": duration_ms,
                "client_ip": client_ip,
                "error_type": type(exc).__name__,
            }
            try:
                self._append_log(log_entry)
            except Exception:
                pass
            raise
def read_audit_logs(log_file: Optional[Path] = None) -> List[Dict[str, Any]]:
    """Read all structured audit log entries from the specified or default file."""
    path = Path(log_file) if log_file is not None else AUDIT_LOG_FILE
    if not path.is_file():
        return []
    with _file_lock:
        content = path.read_text(encoding="utf-8")
    entries = []
    for line in content.splitlines():
        line = line.strip()
        if line:
            entries.append(json.loads(line))
    return entries
"""Cache public Crossref metadata; flag mismatches without inventing citations."""
import argparse
from difflib import SequenceMatcher
import json
from pathlib import Path
import re
import time
import unicodedata
import urllib.parse
import urllib.request
from research_utils import ROOT,write_json,utc_now
def entries(text):
    blocks=[]
    for m in re.finditer(r'@\w+\s*\{([^,]+),',text):
        level=1;i=m.end()
        while i<len(text) and level:
            level+=(text[i]=='{')-(text[i]=='}');i+=1
        body=text[m.end():i-1];fields={}
```

```
// ==================== [第 50 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
        for field in ['title','doi','year','author']:
            f=re.search(r'\b'+field+r'\s*=\s*\{',body,re.I)
            if f:
                start=f.end();j=start;depth=1
                while j<len(body) and depth:
                    depth+=(body[j]=='{')-(body[j]=='}');j+=1
                fields[field]=' '.join(body[start:j-1].split())
        blocks.append({'key':m.group(1),'text':text[m.start():i],**fields})
    return blocks
def normalized(s):return re.sub(r'[^a-z0-9]+','',unicodedata.normalize('NFKD',s.lower()))
def get_metadata(doi,folder):
    name=re.sub(r'[^a-zA-Z0-9]+','_',doi)+'.json';path=folder/name
    if path.exists():return json.loads(path.read_text(encoding='utf-8'))
    url='https://api.crossref.org/works/'+urllib.parse.quote(doi,safe='/')
    for attempt in range(3):
        try:
            req=urllib.request.Request(url,headers={'User-Agent':'SafeAMP-citation-audit/1.0'})
            with urllib.request.urlopen(req,timeout=20) as r:payload=json.load(r)['message']
            write_json(path,{'url':url,'retrieved_at':utc_now(),'metadata':payload})
            return json.loads(path.read_text(encoding='utf-8'))
        except Exception:
            if attempt==2:raise
            time.sleep(2*(attempt+1))
def audit(bib,output):
    output=Path(output);output.mkdir(parents=True,exist_ok=True);rows=[]
    for entry in entries(Path(bib).read_text(encoding='utf-8-sig')):
        row={'key':entry['key'],'expected_title':entry.get('title'),'doi':entry.get('doi'),
             'claim_support':'not_checked','status':'no_doi_requires_primary_source_check'}
        if entry.get('doi'):
            try:
                response=get_metadata(entry['doi'],output/'crossref');actual=response['metadata']
                title=actual.get('title',[''])[0]
                similarity=SequenceMatcher(None,normalized(entry.get('title','')),normalized(title)).ratio()
                expected_authors=[normalized(x.split(',')[0]) for x in entry.get('author','').split(' and ') if
x.strip(
                resolved_authors=[normalized(x.get('family',x.get('name',''))) for x in actual.get('author',[])]
                authors_match=bool(expected_authors) and
expected_authors==resolved_authors[:len(expected_authors)]
                years={str(actual[k]['date-parts'][0][0]) for k in ['published','published-print','published-
online','is
                year_match=entry.get('year') in years
                text_quality='replacement_character_requires_publisher_check' if '\ufffd' in title else 'ok'

row.update(resolved_title=title,title_similarity=similarity,metadata_text_quality=text_quality,metadata_
                           author_metadata_match=authors_match,year_metadata_match=year_match,
                           status='metadata_match_pending_claim_review' if similarity>=.98 and authors_match and
year_ma
            except Exception as e:row.update(status='lookup_failed',reason=str(e))
        rows.append(row)
        write_json(output/'reference_audit.json',
{'checked_at':utc_now(),'all_claims_verified':False,'entries':rows})
        print(entry['key'],row['status'],flush=True)
        time.sleep(.2)
    return rows
if __name__=='__main__':
    p=argparse.ArgumentParser(description=__doc__)
    p.add_argument('--bib',type=Path,default=ROOT/'04_paper/references.bib')
    p.add_argument('--output',type=Path,default=ROOT/'04_paper/evidence')
```

```
// ==================== [第 51 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
    a=p.parse_args();audit(a.bib,a.output.resolve())
"""Build unified and strictly isolated development and external cohorts.
Combines:
- Development cohort: DBAASP v1 (417 experimental records)
- External cohort: Separately sourced experimental records with full assay provenance
Applies:
1. Exact sequence deduplication against development cohort.
2. Literature / PMID / DOI overlap checking against development cohort.
3. Orthogonal homology isolation under CD-HIT standard (Protocol Amendment 01).
4. Full provenance tracking, raw file checksums, and exclusion auditing.
"""
import argparse
import json
from pathlib import Path
from typing import Optional
import numpy as np
import pandas as pd
from feature_extractor import clean_sequence
from cdhit_cluster import cdhit_cluster_sequences
from research_data import sequence_identity, validate_records, REQUIRED, TASKS
from research_utils import ROOT, sha256, utc_now, write_json
def extract_dev_pmids_and_dois(dev_df: pd.DataFrame) -> tuple[set[str], set[str]]:
    pmids, dois = set(), set()
    for ref_str in dev_df["evidence_reference"].dropna():
        try:
            articles = json.loads(ref_str) if isinstance(ref_str, str) else []
            for art in articles:
                pubmed = art.get("pubmed") if isinstance(art.get("pubmed"), dict) else {}
                pmid = pubmed.get("pubmedId") or art.get("pubmedId") or art.get("pmid")
                if pmid:
                    pmids.add(str(pmid).strip())
                doi = art.get("doi")
                if doi:
                    dois.add(str(doi).lower().strip())
        except Exception:
            pass
    return pmids, dois
def extract_record_pmids_and_dois(row) -> tuple[set[str], set[str]]:
    pmids, dois = set(), set()
    ref_str = getattr(row, "evidence_reference", None)
    if ref_str and not pd.isna(ref_str):
        try:
            articles = json.loads(ref_str) if isinstance(ref_str, str) else []
            for art in articles:
                if isinstance(art, dict):
                    pubmed = art.get("pubmed") if isinstance(art.get("pubmed"), dict) else {}
                    pmid = pubmed.get("pubmedId") or art.get("pubmedId") or art.get("pmid")
                    if pmid:
                        pmids.add(str(pmid).strip())
                    doi = art.get("doi")
                    if doi:
                        dois.add(str(doi).lower().strip())
```

```
// ==================== [第 52 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
            except Exception:
                pass
        url = getattr(row, "source_url", "")
        if url and "doi.org/" in str(url):
            doi_part = str(url).split("doi.org/")[-1].strip().lower()
            if doi_part:
                dois.add(doi_part)
        return pmids, dois
    def build_cohorts(
        dev_records_path: Path,
        output_dir: Path,
        external_records_path: Optional[Path] = None,
        cutoff: float = 0.40,
        identity_mode: str = "max",
    ) -> pd.DataFrame:
        if external_records_path is None:
            raise ValueError("An external candidate records file must be explicitly provided via --external;
handwritten def
        output_dir = Path(output_dir)
        output_dir.mkdir(parents=True, exist_ok=True)
        # 1. Load Development Cohort
        dev_records_path = Path(dev_records_path)
        dev_df = pd.read_csv(dev_records_path)
        validate_records(dev_df, verify_raw=True)
        dev_seqs = list(set(dev_df["seq"].map(clean_sequence)))
        dev_pmids, dev_dois = extract_dev_pmids_and_dois(dev_df)
        # Compute CD-HIT representatives of development cohort
        _, dev_reps, _ = cdhit_cluster_sequences(dev_seqs, cutoff=cutoff, min_length_ratio=0.50)
        print(f"[Build Cohorts] Development cohort loaded: {len(dev_df)} records, {len(dev_reps)} CD-HIT cluster
representat
        # 2. Load and Validate External Cohort
        external_records_path = Path(external_records_path)
        if not external_records_path.is_file():
            raise FileNotFoundError(f"External records file not found: {external_records_path}")
        ext_candidates = pd.read_csv(external_records_path)
        validate_records(ext_candidates, verify_raw=True)
        external_records = []
        exclusions = []
        for row in ext_candidates.itertuples():
            eid = getattr(row, "record_id")
            try:
                seq = clean_sequence(getattr(row, "seq"))
            except Exception:
                exclusions.append({"id": eid, "reason": "noncanonical_amino_acid"})
                continue
            if not (10 <= len(seq) <= 50):
                exclusions.append({"id": eid, "reason": "length_out_of_range"})
                continue
            if seq in dev_seqs:
                exclusions.append({"id": eid, "reason": "exact_sequence_overlap_with_development"})
                continue
            # Check literature overlap against development
            row_pmids, row_dois = extract_record_pmids_and_dois(row)
            pmid_overlap = row_pmids & dev_pmids
```

// ==================== [第 53 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================

```
        doi_overlap = row_dois & dev_dois
        if pmid_overlap or doi_overlap:
            exclusions.append({
                "id": eid,
                "reason": f"literature_overlap_with_development (pmid={pmid_overlap}, doi={doi_overlap})"
            })
            continue
        # Check homology isolation against development
        if identity_mode == "max":
            max_id = max(
                (sequence_identity(seq, d, identity_mode="max") for d in dev_seqs),
                default=0.0
            )
        else:
            max_id = max(
                (sequence_identity(seq, r) for r in dev_reps
                 if len(seq)/len(r) >= 0.50 and len(r)/len(seq) >= 0.50),
                default=0.0
            )
        if max_id >= cutoff:
            exclusions.append({"id": eid, "reason": f"homology_cross_isolation_violation_{max_id:.3f}"})
            continue
        external_records.append({col: getattr(row, col) for col in REQUIRED})
    ext_df = pd.DataFrame(external_records, columns=REQUIRED)
    print(f"[Build Cohorts] External cohort curated: {len(ext_df)} records (AMP pos={(ext_df['amp']==1).sum()},
neg={(ex
    # Combine development and external cohorts
    combined_df = pd.concat([dev_df, ext_df], ignore_index=True)
    combined_records_path = output_dir / "records.csv"
    combined_df.to_csv(combined_records_path, index=False)
    write_json(output_dir / "exclusions.json", exclusions)
    manifest = {
        "status": "cohorts_built",
        "created_at": utc_now(),
        "total_records": len(combined_df),
        "development_records": len(dev_df),
        "external_records": len(ext_df),
        "cutoff": cutoff,
        "identity_mode": identity_mode,
        "development_cdhit_clusters": len(dev_reps),
        "records_sha256": sha256(combined_records_path),
        "external_exclusions": len(exclusions),
    }
    write_json(output_dir / "manifest.json", manifest)
    print(f"[Build Cohorts] Combined dataset written to {combined_records_path} (Total: {len(combined_df)}
records).")
    return combined_df
if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--dev", type=Path, required=True)
    p.add_argument("--external", type=Path, required=True)
    p.add_argument("--output", type=Path, required=True)
    p.add_argument("--cutoff", type=float, default=0.40)
    p.add_argument("--identity-mode", type=str, default="max")
```

```
// ==================== [第 54 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
    a = p.parse_args()
    build_cohorts(a.dev, a.output, external_records_path=a.external, cutoff=a.cutoff,
identity_mode=a.identity_mode)
"""Create an immutable candidate source without unsupported external labels."""
import argparse
from pathlib import Path
import pandas as pd
from evidence_validation import validate_source_evidence
from research_data import ResearchDataError, validate_records
from research_utils import sha256, utc_now, write_json
def curate_evidence_clean_source(records_path: Path, output_dir: Path) -> dict:
    records_path, output_dir = Path(records_path), Path(output_dir)
    if output_dir.exists():
        raise ValueError('Output exists; use a new versioned source directory.')
    records = validate_records(pd.read_csv(records_path), verify_raw=False)
    try:
        validate_source_evidence(records)
        failures = []
    except ResearchDataError as exc:
        if not exc.evidence_audit:
            raise
        failures = exc.evidence_audit['failures']
    failed_ids = {failure['record_id'] for failure in failures}
    if any(record_id.startswith('RAW_FILE:') for record_id in failed_ids):
        raise ResearchDataError('Raw source integrity failure cannot be repaired by excluding records.')
    if any(records.set_index('record_id').loc[record_id, 'source_role'] != 'external' for record_id in
failed_ids):
        raise ResearchDataError('Development evidence failure requires source review, not automatic exclusion.')
    accepted = records[~records.record_id.isin(failed_ids)].copy()
    accepted_audit = validate_source_evidence(accepted)
    output_dir.mkdir(parents=True)
    accepted.to_csv(output_dir / 'records.csv', index=False)
    write_json(output_dir / 'exclusions.json', failures)
    write_json(output_dir / 'evidence_audit.json', accepted_audit)
    manifest = {
        'status': 'candidate' if accepted.source_role.eq('external').any() else 'blocked_no_external_cohort',
        'created_at': utc_now(),
        'input_records': str(records_path.resolve()),
        'input_records_sha256': sha256(records_path),
        'records_sha256': sha256(output_dir / 'records.csv'),
        'exclusions_sha256': sha256(output_dir / 'exclusions.json'),
        'evidence_audit_sha256': sha256(output_dir / 'evidence_audit.json'),
        'development_records': int(accepted.source_role.eq('development').sum()),
        'external_records': int(accepted.source_role.eq('external').sum()),
        'excluded_unsupported_external_records': len(failures),
    }
    write_json(output_dir / 'source_manifest.json', manifest)
    return manifest
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--records', type=Path, required=True)
    parser.add_argument('--output', type=Path, required=True)
    args = parser.parse_args()
    print(curate_evidence_clean_source(args.records, args.output))
```

```
// ==================== [第 55 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
"""Provenance validation and all-member homology-separated research partitions.
The alignment is an explicitly defined Needleman-Wunsch implementation, not CD-HIT.
Connected components keep every above-cutoff pair in the same partition. A giant
component or insufficient class coverage is a failed feasibility check, not license
to relax the cutoff or silently discard observations.
"""
import argparse
from collections import Counter
import hashlib
import json
import re
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedGroupKFold
from feature_extractor import clean_sequence
from research_utils import ROOT, sha256, write_json, utc_now
TASKS = ('amp', 'hemo')
REQUIRED = ('record_id', 'seq', 'amp', 'hemo', 'source_dataset', 'source_accession',
            'source_url', 'evidence_reference', 'label_basis', 'assay_conditions',
            'modifications', 'evidence_type', 'is_synthetic', 'source_role', 'raw_path', 'raw_sha256')
class ResearchDataError(ValueError):
    def __init__(self, message, evidence_audit=None):
        super().__init__(message)
        self.evidence_audit = evidence_audit
def validate_records(frame, verify_raw=True):
    missing = set(REQUIRED) - set(frame.columns)
    if missing:
        raise ResearchDataError('Missing provenance columns: ' + ', '.join(sorted(missing)))
    df = frame.copy()
    if df.empty or df.record_id.isna().any() or df.record_id.duplicated().any():
        raise ResearchDataError('Record identifiers must be present, unique and nonempty.')
    for col in set(REQUIRED) - set(TASKS):
        if df[col].isna().any() or df[col].astype(str).str.strip().eq('').any():
            raise ResearchDataError(f'Missing evidence in {col}.')
    if not df.is_synthetic.astype(str).str.lower().isin(['false', '0']).all():
        raise ResearchDataError('Synthetic records cannot enter research evaluation.')
    if not df.evidence_type.eq('experimental').all():
        raise ResearchDataError('Only experimental evidence is accepted; unannotated is not negative.')
    if not df.modifications.eq('none').all():
        raise ResearchDataError('Modified peptides need a separately specified representation.')
    if not df.source_role.isin(['development', 'external']).all():
        raise ResearchDataError('Unknown source role.')
    for task in TASKS:
        df[task] = pd.to_numeric(df[task], errors='raise')
        if not df[task].dropna().isin([0, 1]).all():
            raise ResearchDataError('Observed labels must be binary; unknown labels must be blank.')
        observed=df[task].notna()
        if task+'_observed' in df and not
df[task+'_observed'].astype(str).str.lower().eq('true').equals(observed):
            raise ResearchDataError('Observed-label flag disagrees with missing label state.')
        df[task+'_observed']=observed
    if df[list(TASKS)].isna().all(axis=1).any():
```

```
// ==================== [第 56 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
            raise ResearchDataError('Every included record needs at least one measured endpoint.')
        df['seq'] = df.seq.map(clean_sequence)
        if not df.seq.str.len().between(10, 50).all():
            raise ResearchDataError('Prespecified research domain is 10–50 canonical residues.')
        if verify_raw:
            from evidence_validation import validate_source_evidence
            validate_source_evidence(df)
        return df
    def consolidate_records(df):
        """Merge identical molecules; conflicting measured labels become unknown."""
        rows, excluded = [], []
        for seq, group in df.groupby('seq', sort=True):
            labels = {}
            conflicts = []
            for task in TASKS:
                observed = group[task].dropna().unique()
                labels[task] = float(observed[0]) if len(observed) == 1 else np.nan
                if len(observed) > 1:
                    conflicts.append(task)
            if len(set(group.source_role)) > 1:
                excluded.append({'record_ids': group.record_id.tolist(), 'reason': 'exact development/external
overlap'})
                continue
            if all(pd.isna(v) for v in labels.values()):
                excluded.append({'record_ids': group.record_id.tolist(), 'reason': 'no unambiguous label'})
                continue
            rows.append({'sample_id': hashlib.sha256(seq.encode()).hexdigest()[:24], 'seq': seq, **labels,
                         **{task+'_observed':not pd.isna(labels[task]) for task in TASKS},
                         'source_role': group.source_role.iloc[0],
                         'source_datasets': '|'.join(sorted(set(group.source_dataset))),
                         'record_ids': json.dumps(group.record_id.tolist()), 'conflicts': '|'.join(conflicts)})
        return pd.DataFrame(rows), excluded
    def sequence_length_ratio(a: str, b: str) -> float:
        if not a or not b:
            raise ResearchDataError('Empty sequence cannot have length ratio.')
        return min(len(a), len(b)) / max(len(a), len(b))
    def sequence_identity(a, b, *, identity_mode='min'):
        """Global NW identity with a frozen traceback and explicit denominator."""
        if not a or not b:
            raise ResearchDataError('Empty sequence cannot be aligned.')
        if identity_mode not in ('min', 'max', 'alignment'):
            raise ResearchDataError(f"Unknown identity_mode: '{identity_mode}', expected 'min', 'max' or
'alignment'.")
        # Canonical orientation makes traceback tie handling symmetric.
        if a > b:
            a, b = b, a
        n, m = len(a), len(b)
        # Track the traceback-equivalent number of matches, preferring diagonal ties.
        prev = [-2*j for j in range(m+1)]
        pm = [0]*(m+1)
        pl = list(range(m+1))
        for i, aa in enumerate(a, 1):
            cur, cm = [-2*i] + [0]*m, [0]*(m+1)
            cl = [i] + [0]*m
```

```
// ==================== [第 57 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
            for j, bb in enumerate(b, 1):
                diag = prev[j-1] + (2 if aa == bb else -1)
                up, left = prev[j]-2, cur[j-1]-2
                if diag >= up and diag >= left:
                    cur[j], cm[j] = diag, pm[j-1] + (aa == bb)
                    cl[j] = pl[j-1] + 1
                elif up >= left:
                    cur[j], cm[j] = up, pm[j]
                    cl[j] = pl[j] + 1
                else:
                    cur[j], cm[j] = left, cm[j-1]
                    cl[j] = cl[j-1] + 1
            prev, pm, pl = cur, cm, cl
        matches = pm[-1]
        denom = {'min': min(n, m), 'max': max(n, m), 'alignment': pl[-1]}[identity_mode]
        return matches / denom
    def homology_components(sequences, cutoff=0.40, *, identity_mode='min'):
        if not 0 < cutoff <= 1:
            raise ResearchDataError('Identity cutoff must be in (0, 1].')
        parent = list(range(len(sequences)))
        counts = [Counter(s) for s in sequences]
        def find(i):
            while parent[i] != i:
                parent[i] = parent[parent[i]]
                i = parent[i]
            return i
        if identity_mode not in ('min', 'max', 'alignment'):
            raise ResearchDataError(f'Unknown identity mode: {identity_mode}')
        checked, edges = 0, 0
        for i, a in enumerate(sequences):
            for j in range(i):
                denom = min(len(a), len(sequences[j])) if identity_mode == 'min' else max(len(a), len(sequences[j]))
                if sum((counts[i] & counts[j]).values()) / denom < cutoff:
                    continue
                checked += 1
                if sequence_identity(a, sequences[j], identity_mode=identity_mode) >= cutoff:
                    parent[find(i)] = find(j)
                    edges += 1
        roots = [find(i) for i in range(len(parent))]
        ids = {v: k for k, v in enumerate(sorted(set(roots)))}
        return np.array([ids[r] for r in roots]), {
            'possible_pairs': len(sequences) * (len(sequences) - 1) // 2,
            'aligned_pairs': checked, 'above_cutoff_pairs': edges, 'groups': len(ids),
            'identity_mode': identity_mode,
        }
    def source_study_ids(row):
        """Extract DOI/PMID identifiers from a record's declared primary references."""
        refs = str(row.evidence_reference)
        identifiers = {f'doi:{doi.lower().rstrip(".,;)}")}' for doi in re.findall(r'10\.\d{4,9}/[^\s"<>]+', refs,
re.I)}
        identifiers.update(f'pmid:{pmid}' for pmid in re.findall(r'PMID\s*[:=]\s*(\d+)', refs, re.I))
        try:
            payload = json.loads(refs)
```

```
// ==================== [第 58 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
    except (TypeError, ValueError):
        payload = None
    def visit(value):
        if isinstance(value, dict):
            for key, item in value.items():
                if key.lower() == 'doi' and item:
                    identifiers.add(f'doi:{str(item).lower().strip()}')
                elif key.lower() in ('pubmedid', 'pmid') and item:
                    identifiers.add(f'pmid:{str(item).strip()}')
                visit(item)
        elif isinstance(value, list):
            for item in value:
                visit(item)
    visit(payload)
    return identifiers
def assert_study_independence(records):
    """External records must not reuse a development cohort's primary study."""
    dev_studies = set().union(*(source_study_ids(row) for row in
records[records.source_role.eq('development')].itertupl
    for row in records[records.source_role.eq('external')].itertuples():
        overlap = source_study_ids(row) & dev_studies
        if overlap:
            raise ResearchDataError(f'External record {row.record_id} shares a development study:
{sorted(overlap)}')
def analyze_homology(records, spec):
    """Build all-member components and retain every development/external pair score."""
    df, exclusions = consolidate_records(records)
    if df.empty:
        raise ResearchDataError('No eligible molecules after consolidation.')
    if spec.clustering_method == 'transitive':
        cluster_ids, summary = homology_components(
            df.seq.tolist(), spec.cutoff, identity_mode=spec.membership_identity
        )
        df['cluster_id'] = cluster_ids
        df['representative_sample_id'] = df['sample_id']
    elif spec.clustering_method == 'greedy_nw':
        from cdhit_cluster import cdhit_cluster_sequences
        cluster_ids, reps, summary = cdhit_cluster_sequences(
            df.seq.tolist(), cutoff=spec.cutoff, min_length_ratio=spec.min_length_ratio
        )
        df['cluster_id'] = cluster_ids
        rep_map = {c_idx: df.loc[df.seq.eq(rep_seq), 'sample_id'].iloc[0] for c_idx, rep_seq in enumerate(reps)}
        df['representative_sample_id'] = df['cluster_id'].map(rep_map)
    else:
        raise ResearchDataError(f'Unknown clustering method: {spec.clustering_method}')
    pairs = []
    dev = df[df.source_role.eq('development')]
    ext = df[df.source_role.eq('external')]
    for a in ext.itertuples():
        for b in dev.itertuples():
            identity = sequence_identity(a.seq, b.seq, identity_mode=spec.cross_identity)
            pairs.append({
                'external_sample_id': a.sample_id,
                'development_sample_id': b.sample_id,
```

```
// ==================== [第 59 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
                'identity': identity,
                'violates_cutoff': identity >= spec.cutoff,
            })
    cross_pairs = pd.DataFrame(pairs, columns=[
        'external_sample_id', 'development_sample_id', 'identity', 'violates_cutoff'
    ])
    audit = {
        'protocol_id': spec.protocol_id,
        'identity_mode': spec.cross_identity,
        'cutoff': spec.cutoff,
        'development_samples': len(dev),
        'external_samples': len(ext),
        'cross_pairs_evaluated': len(cross_pairs),
        'cross_pair_violations': int(cross_pairs.violates_cutoff.sum()),
        'clustering_summary': summary,
        'largest_cluster': int(df.groupby('cluster_id').size().max()),
    }
    return df, exclusions, cross_pairs, audit
def assert_cross_isolation(left, right, cutoff=0.40, *, protocol=None):
    if protocol is None:
        mode = 'min'
        scope = 'all_members'
        min_len_ratio = 0.0
    else:
        if isinstance(protocol, str):
            from research_protocol_config import get_protocol
            spec = get_protocol(protocol)
        else:
            spec = protocol
        mode = spec.cross_identity
        scope = spec.cross_scope
        cutoff = spec.cutoff
        min_len_ratio = spec.min_length_ratio
    if scope == 'all_members':
        for a in left.itertuples():
            ca = Counter(a.seq)
            for b in right.itertuples():
                if min_len_ratio > 0 and sequence_length_ratio(a.seq, b.seq) < min_len_ratio:
                    continue
                denom = min(len(a.seq), len(b.seq)) if mode == 'min' else max(len(a.seq), len(b.seq))
                if sum((ca & Counter(b.seq)).values()) / denom < cutoff:
                    continue
                sim = sequence_identity(a.seq, b.seq, identity_mode=mode)
                if sim >= cutoff:
                    raise ResearchDataError(f'Cross-partition sequence similarity violates {cutoff}:
{a.sample_id} / {b.
    elif scope == 'members_to_representatives':
        left_reps = left[left.sample_id == left.representative_sample_id] if 'representative_sample_id' in left
else lef
        right_reps = right[right.sample_id == right.representative_sample_id] if 'representative_sample_id' in
right els
        for a in left.itertuples():
            ca = Counter(a.seq)
            for r in right_reps.itertuples():
                if min_len_ratio > 0 and sequence_length_ratio(a.seq, r.seq) < min_len_ratio:
```

```
// ==================== [第 60 页 / 共 60 页] 后端计算引擎与双任务模型模块 ====================
                    continue
                denom = min(len(a.seq), len(r.seq)) if mode == 'min' else max(len(a.seq), len(r.seq))
                if sum((ca & Counter(r.seq)).values()) / denom < cutoff:
                    continue
                sim = sequence_identity(a.seq, r.seq, identity_mode=mode)
                if sim >= cutoff:
                    raise ResearchDataError(f'Cross-partition sequence similarity violates {cutoff}:
{a.sample_id} / rep
def require_classes(df, context):
    for task in TASKS:
        if set(df[task].dropna().unique()) != {0, 1}:
            raise ResearchDataError(f'{context}: both measured classes required for {task}.')
def partition(df, folds=5, seed=42):
    if df.empty:
        raise ResearchDataError('No eligible experimental records.')
    out = df.copy()
    dev = out[out.source_role.eq('development')]
    ext = out[out.source_role.eq('external')]
    if ext.empty:
        raise ResearchDataError('A separately sourced external cohort is required.')
    col = 'source_datasets' if 'source_datasets' in dev else ('source_dataset' if 'source_dataset' in dev else
None)
    if col:
        dev_sources = set('|'.join(dev[col].astype(str)).split('|'))
        ext_sources = set('|'.join(ext[col].astype(str)).split('|'))
        if dev_sources & ext_sources:
            raise ResearchDataError('External source cannot also supply development records.')
    if set(dev.cluster_id) & set(ext.cluster_id):
        raise ResearchDataError('Development/external cohorts contain homologous sequences.')
    if len(set(dev.cluster_id)) < folds+1:
        raise ResearchDataError('Insufficient independent homology groups for five-fold development plus
holdout.')
    strata = dev.amp.fillna(-1).astype(int).astype(str) + '_' + dev.hemo.fillna(-1).astype(int).astype(str)
    outer = StratifiedGroupKFold(n_splits=folds+1, shuffle=True, random_state=seed)
    development_idx, test_idx = next(outer.split(dev, strata, dev.cluster_id))
    development, test = dev.iloc[development_idx], dev.iloc[test_idx]
    require_classes(development, 'Development'); require_classes(test, 'Independent test'); require_classes(ext,
'Extern
    out['split'], out['fold'] = 'external', -1
    out.loc[test.index, 'split'] = 'test'
    out.loc[development.index, 'split'] = 'development'
    labels = development.amp.fillna(-1).astype(int).astype(str) + '_' +
development.hemo.fillna(-1).astype(int).astype(s
    cv = StratifiedGroupKFold(n_splits=folds, shuffle=True, random_state=seed)
    for fold, (a, b) in enumerate(cv.split(development, labels, development.cluster_id)):
        require_classes(development.iloc[a], f'Fold {fold} training'); require_classes(development.iloc[b],
f'Fold {fold
        out.loc[development.iloc[b].index, 'fold'] = fold
    return out
def validate_partition(df, cutoff=.40, *, protocol=None):
    if df.loc[df.split.eq('external'),'source_role'].ne('external').any() or
df.loc[df.split.ne('external'),'source_role
        raise ResearchDataError('Prepared split changed the declared source role.')
    if set(df.split) != {'development', 'test', 'external'}:
        raise ResearchDataError('Development, test and external partitions are required.')
    if set(df.loc[df.split.eq('development'), 'fold']) != set(range(5)):
        raise ResearchDataError('Exactly five development folds are required.')
    if df.sample_id.duplicated().any() or df.seq.duplicated().any():
        raise ResearchDataError('Duplicate molecule in prepared partitions.')
```